Certificando a ausência de efeito e identificando a condição de vitória sob pontuação por ação no ARC-AGI-3

Julio Cesar Silveira de Souza

Tradução do original em inglês ("Certifying No Effect…", versão submetida ao ARC Prize 2026 Paper Track em 8 de novembro de 2026), produzida com o auxílio de agentes de modelos de linguagem e lida pelo autor. As figuras mantêm os rótulos em inglês. Onde as versões diferirem, vale a versão em inglês.

Resumo

O ARC-AGI-3 avalia a definição de objetivos, a identificação de estados desejáveis sem instruções; no entanto, cada jogo fixa a sua condição de vitória, e a pontuação mede com que economia ela é alcançada. Pela taxonomia de Ackoff e Emery, os agentes que competem são, no máximo, buscadores de múltiplos objetivos. Nossa definição de trabalho de uma inteligência geral, mais larga do que a AGI que o benchmark define, é a de um sistema com propósito, que escolhe tanto os objetivos quanto os meios, e estudamos o passo que está ao alcance dentro do benchmark: identificar o objetivo de um jogo sem gastar as ações que a pontuação cobra. Nosso agente acrescenta ao *harness* de um grande modelo de linguagem (LLM) de pesos abertos um explorador baseado em grafo que emite certificados revogáveis de ausência de efeito, detecta estado oculto quando um deles falha ou quando uma ação segura mata, e ordena hipóteses de objetivo que apenas ordenam ações. Com o LLM ainda como política padrão, a camada é um instrumento de medida cuja presença variamos. Com a camada jogando, nossa primeira submissão pontuada chegou a 0,75 sobre outra base, contra 26,74 da base dfranzen sem modificação, sem pareamento, mas compatível com o custo das ações. Aconselhando por notas no prompt e poda da lista, a camada não teve efeito mensurável em duas ablações de três rodadas, com cinco dias de intervalo, com o limiar fixado depois de um piloto de uma rodada. No modo de último recurso, uma tiragem no conjunto oculto, feita depois do prazo, deu 28,41 contra 26,74, dentro da dispersão da base. Com as famílias de objetivo padrão, 50 de 66 vitórias locais, não nomeadas pelo identificador, não tinham nenhum predicado implementado verdadeiro no quadro vencedor. Relatamos uma lacuna medida na definição de objetivos, um dos quatro componentes do benchmark, e uma direção além desta competição.

1 Introdução

O ARC-AGI-3 é o terceiro benchmark da série ARC-AGI e o primeiro inteiramente interativo, ao passo que o ARC-AGI-1 (2019) e o ARC-AGI-2 (2025) consistiam em tarefas estáticas em grade (ARC Prize Foundation, 2026, p. 1; Kamradt, 2026a). O ARC-AGI-3 coloca um agente dentro de um jogo inédito, por turnos, cujas regras, mecânicas e condições de vitória nunca são enunciadas, e o agente inspeciona uma grade de 64×64 e escolhe num conjunto discreto e limitado de ações. É avaliado pela Eficiência Relativa de Ações Humanas (RHAE, *Relative Human Action Efficiency*), que toma `min(1.15, (h/a)^2)` por nível contra uma linha de base humana de primeira execução, na mediana superior, ponderada pelo índice do nível. O artigo que apresenta o benchmark nomeia quatro componentes da inteligência agêntica que ele avalia: exploração, modelagem, definição de objetivos, e planejamento e execução. No lançamento, os sistemas de fronteira pontuaram abaixo

de 1%, e humanos resolveram todos os ambientes (ARC Prize Foundation, 2026, pp. 1, 5, 6, 12, 19). Em setembro, no conjunto semiprivado, um modelo de fronteira pontuou 62,7% sob o *harness* padrão da fundação, com esforço máximo de raciocínio, e 99,9% com esforço alto num *harness* que preserva o estado de raciocínio do seu provedor, onde, com esforço máximo, usou menos ações do que a mediana humana em 96% dos níveis (Kamradt, 2026b). Este artigo, em vez disso, trabalha nas condições da competição, sem conexão, sem API de modelo e com limite de nove horas, onde nosso agente roda um modelo de pesos abertos numa única GPU (Seção 3.1).

Uma linha recente de trabalhos constrói modelos de mundo executáveis e os verifica contra o histórico de interação (Rodionov, 2026, p. 1; Courtis et al., 2026, p. 2; Skoutnev et al., 2026, p. 3; Lehmann et al., 2026, p. 1). Dois deles, descritos na Seção 2, medem o seu gargalo, o objetivo. No Twin, a dinâmica é aprendível, ao passo que o predicado do objetivo ainda não é enunciado com confiabilidade (Skoutnev et al., 2026, p. 28). No Tycho, a política com os modelos de transição mais exatos pontua abaixo de outra política na mesma comparação, 83,07 contra 88,49 RHAE (Lehmann et al., 2026, pp. 1, 24). Nesses sistemas, como o mundo se move é cada vez mais certificado, mas o que contaria como vencer ainda não está estabelecido com confiabilidade. Tampouco, do modo como formulamos o problema, está estabelecido quais ações podem ser desconsideradas com segurança enquanto se descobre. Uma equipe sétima colocada na competição mediu a mesma lacuna pelo outro lado: dadas as regras escritas de cada jogo público e uma dica por nível, o seu modelo "cleared 70% more levels on the hardest games and scored 85.7 instead of 64.4 on the 25 public games", de modo que "most of the gap is in working that out" (Sestak et al., 2026). O modelo de fronteira mencionado acima seguiu um caminho aparentado em suas notas, construindo «modelos de mundo simbólicos compactos» (Kamradt, 2026b). Uma lacuna de descoberta persiste sem o canal visual, como no DiG-bench (Battleday et al., 2026, p. 6), 70 jogos de texto cujas regras e condições de vitória são ocultas, onde o Gemini 3.1 Pro venceu 18 jogos, e 69 quando recebeu uma breve descrição da dinâmica e da condição de vitória. Onde havia comparação com o mesmo modelo, os *harnesses* agênticos ali testados, entre eles o PRO-LONG (Fox et al., 2026) e o Prime Agent, não se saíram melhor do que o básico nos dois níveis mais difíceis.

Esses jogos são construídos e testados por humanos, e cada um entra no benchmark só se pelo menos duas das dez pessoas que o tentam primeiro o resolvem por inteiro (ARC Prize Foundation, 2026, pp. 7, 16). Eles se apoiam apenas nos *priors* de conhecimento nuclear (*Core Knowledge*, p. 8), os sistemas de representação de objetos, agentes, número e espaço que Spelke e Kinzler (2007, p. 89) descrevem. Tolman (1948, pp. 198–199) observou que, em experimentos de discriminação, os sujeitos humanos chegam com as instruções, ao passo que seus ratos «não sabem o que se quer deles», de modo que a maior parte do seu aprendizado parece consistir em descobrir as instruções. O ARC-AGI-3 retém as instruções de todo jogador, e o agente «deve inferir autonomamente» a condição de vitória (ARC Prize Foundation, 2026, p. 5).

Na ausência de instruções, o lance mais informativo é com frequência uma sonda, uma ação escolhida para distinguir entre duas hipóteses concorrentes sobre uma das regras do jogo (por exemplo, se tocar um objeto o move ou o remove), mesmo que se desvie do objetivo aparente. Avaliada pelo progresso rumo a esse objetivo, tal sonda parece um retrocesso. Lehman e Stanley (2011, pp. 190, 193) chamam de enganosos os problemas desse tipo, cujo sinal de progresso aponta para longe da solução. Conjecturamos que uma política conduzida por um grande modelo de linguagem (LLM) propõe tais sondas menos do que deveria, ao menos um modelo de pesos abertos do tamanho que a competição permite. O prompt de sistema da base dfranzen sobre a qual construímos (Franzen, 2026a; Seção 3.1) já diz ao modelo: «Se a confiança for baixa, programe uma sonda discriminante». Ainda assim, o modelo amostra o que seu treinamento torna provável, e raciocina dentro de regras

literalizadas a partir de poucas transições. No nosso sistema, o sinal potencialmente enganoso é explícito: o escalar de progresso de cada família de objetivo (Seção 3.3). É por isso que nenhuma hipótese pode proibir uma ação.

Não proibir ação alguma mantém a busca aberta, mas deixa intocada uma pergunta anterior, a saber, quem define o objetivo que a busca persegue, e o benchmark responde a parte dela por construção. A ARC Prize Foundation define a inteligência artificial geral (AGI) como a «capacidade de adquirir qualquer habilidade que um humano pode, com a eficiência de um humano» (ARC Prize Foundation, 2026, p. 4). A definição de objetivos, um dos quatro componentes, é «a capacidade de identificar estados futuros interessantes ou desejáveis sem instruções explícitas», e o agente deve determinar «o que visar» a partir do seu «próprio impulso intrínseco e das pistas do ambiente» (p. 5). A fundação define inteligência como eficiência através desses componentes (p. 5) e lê o progresso rumo à AGI apenas em ambientes nunca vistos, sem *harnesses* específicos de tarefa (p. 15). Chollet, que projetou o benchmark e cofundou a fundação (pp. 1–2), define inteligência como eficiência na aquisição de habilidades e construiu o ARC original para medir «o tipo de inteligência geral» que definiu (Chollet, 2019, pp. 27, 46). A literatura sobre inteligência e sobre AGI responde a essa pergunta de muitas outras maneiras, e não a revisamos aqui. Delimitamos o contraste às definições que o benchmark adota, que pertencem a uma linhagem, postas diante de uma leitura vinda do pensamento sistêmico que inspira nossa abordagem. Essa leitura é a taxonomia de Ackoff e Emery (1972/2017, pp. 30–31), ordenada por quem escolhe o objetivo. Um sistema buscador de objetivo responde a eventos de maneiras diferentes, mas todas as suas respostas servem para produzir um resultado particular, o seu objetivo. Um sistema buscador de múltiplos objetivos é buscador de objetivo em vários ambientes e busca objetivos diferentes em pelo menos dois deles, e é o ambiente, não o sistema, que determina qual, como num computador que joga vários jogos e recebe de «uma fonte externa» a indicação de qual jogar. Um sistema com propósito «pode mudar seus objetivos em condições ambientais constantes», selecionando objetivos além de meios.

Pela nossa leitura dessa linhagem, as habilidades a adquirir e os objetivos a que servem são fixados fora do sistema que está sendo medido, mesmo quando ele não os recebe enunciados. Chollet declara o escopo explicitamente: o ARC «avalia apenas uma forma geral de inteligência fluida, com foco em raciocínio e abstração», num formato deliberadamente próximo dos testes de QI (2019, p. 50). Ele também põe de lado o teste de Turing, que delega o julgamento a juízes humanos, como uma «pista falsa» para o campo (p. 3). Duas leituras de fora dessa linhagem bastam para o nosso contraste. Ackoff e Emery notam a incerteza «sobre o que de fato é medido pelos chamados testes de QI» e relacionam a inteligência à taxa com que um sujeito aprende. Distinguem a inteligência medida pelos testes de QI de uma inteligência criativa que incorpora aspectos de consciência do ambiente e de si, que consideram mais próxima da definição de Chein, em que a relevância é fixada pelos «propósitos imediatos e presumivelmente futuros do ator». Apontam também «a dificuldade de tentar representar a inteligência por um único número» (1972/2017, pp. 52–53). Maturana e Guiloff (1980), a partir da biologia, argumentam que a inteligência não é diretamente observável e «não pode ser medida», estimando um teste de QI no máximo «um subdomínio do domínio de consensualidade entre o observador e o sujeito» (p. 135). Argumentam ainda que um objetivo não é operante na dinâmica própria de um sistema e pertence, em vez disso, à descrição feita por um observador (p. 140).

Lidos em conjunto, os dois vocabulários falam do mesmo objeto. O resultado que as respostas de um sistema servem para produzir, no relato de Ackoff e Emery, é o estado futuro desejável que a definição de objetivos da fundação visa. Definir um objetivo, no sentido da fundação, é identificar o estado a visar sem que lhe digam, e buscá-lo, no sentido de Ackoff e Emery, é agir para produzi-lo. A

taxonomia deles pergunta, além disso, se foi o sistema ou o seu ambiente que fez a determinação, e nessa pergunta nenhuma das duas leituras a coloca no ambiente. Ackoff e Emery ligam a inteligência aos propósitos do ator, e Maturana e Guiloff colocam o objetivo na descrição de um observador. No ARC-AGI-3, o ambiente é o jogo, e suas regras ocultas fixam a condição de vitória que encerra cada nível. A pontuação não é uma dessas regras. A fundação a computa de fora, comparando as ações do agente por nível com as de jogadores humanos nos mesmos jogos, que é a RHAE (pp. 11, 13), de modo que ela mede com que economia cada condição de vitória é alcançada, e não qual objetivo foi escolhido. A referência é uma partida humana de primeira execução, na mediana superior, e não o jogo mais eficiente, que a fundação também estima como uma partida ótima uma vez compreendidos as mecânicas e os objetivos de um jogo (ARC Prize Foundation, 2026, p. 19). Essa referência vem de cerca de 500 membros do público «não selecionados por experiência ou aptidão na resolução de quebra-cabeças» (Kamradt, 2026b). Dentro do ambiente, portanto, definir objetivos equivale a identificar um objetivo que as regras do jogo já determinam, e os agentes que competem, o nosso incluído, são no máximo buscadores de múltiplos objetivos. A Figura 1 põe essa leitura ao lado da classificação comportamental de sistemas que Gharajedaghi (2011, p. 36) extrai de Ackoff, na qual um sistema buscador de objetivo tem escolha de meios, mas não de fins. Tal sistema, se tem memória, «pode aprender a perseguir o seu objetivo com mais eficiência ao longo do tempo» (p. 37), o que uma pontuação por ação recompensa, ao passo que um sistema com propósito «pode mudar os seus fins sob condições constantes» (p. 37).

Behavior Process	Means Structure	End Function	Reading in Section 1
Passive tools	No choice; one structure in all environments	No choice; one function in all environments	
Reactive state-maintaining	No choice; variable but determined	No choice; one function in all different environments	
Responsive goal-seeking	Choice of means; variable and chosen	No choice of ends; variable but determined	The agents of ARC-AGI-3, ours included. Each game's rules fix the end, its win condition; the score charges the means, actions per level against a human reference. Across games, at most multi-goal-seeking: the game, not the agent, sets which goal (Ackoff & Emery).
Active purposeful	Choice of means; variable and chosen	Choice of ends; variable and chosen	The working definition of a general intelligence adopted here: a purposeful system that selects its goals as well as its means. Not tested.

Left three columns: Gharajedaghi (2011, p. 36, Table 2.1), after Ackoff.

Figura 1: Classificação comportamental de sistemas em Gharajedaghi (2011, p. 36, Tabela 2.1), a partir de Ackoff, com a leitura da Seção 1 ao lado. Os agentes do ARC-AGI-3 escolhem os seus meios e não os seus fins, e a pontuação cobra os meios; a definição de trabalho adotada aqui pede também a escolha dos fins.

A instrução que os jogos retêm reaparece a montante. No exemplo de Ackoff e Emery, uma fonte externa diz ao computador qual jogo jogar. Aqui, essa fonte é o desenvolvedor, que escolhe os *priors* do agente, suas ferramentas e, no nosso caso, as famílias de objetivo entre as quais ele busca (Seção 3.3). Os autores do *harness* Duck, sobre o qual as nossas duas bases se constroem, relatam o mesmo do lado deles: para que o agente tivesse bom desempenho, «tivemos de codificar muito

conhecimento específico nos prompts», inclusive «conduzindo a sua definição de objetivos», por exemplo para impedi-lo de tomar a barra de energia como objetivo (Bessis et al., 2026b). Roli, Jaeger e Kauffman (2022), para quem nenhum sistema num quadro algorítmico pode «escolher e definir seus próprios objetivos» (p. 8), colocam a escolha nas mesmas mãos. Os projetistas fixam os critérios de aprendizado, e mesmo funções agnósticas quanto à tarefa são «o resultado de uma escolha externa imposta» (p. 9). A fundação traça a mesma linha do seu lado, já que o ARC-AGI-3 não se destina a medir «a quantidade de inteligência humana que entrou no projeto» de um sistema construído para ele (ARC Prize Foundation, 2026, p. 15). O que o prêmio busca dessa inteligência é, pela nossa leitura, as ideias por trás dos projetos, já que os participantes devem abrir o código das suas soluções para receber o prêmio em dinheiro (p. 21). Quanto da identificação um desenvolvedor pode fornecer é visível nas medições da fundação. Num ambiente público, o mesmo modelo de fronteira pontuou 0,0% sem *harness* e 97,1% com um feito à mão, e 0,0% nos dois casos em outro, o que a fundação lê como evidência de que tais *harnesses* não se transferem a ambientes nunca vistos (p. 15). Sua leitura de AGI, por sua vez, está ligada a um *harness* mínimo e neutro quanto ao provedor: «uma futura AGI deve ser capaz de resolver o ARC-AGI-3 sob essas condições» (Kamradt, 2026b).

Os dois vocabulários medem quão bem os fins são perseguidos, como eficiência através dos componentes do benchmark (ARC Prize Foundation, 2026, p. 5) ou como a taxa com que um sujeito aprende, sendo o sujeito, para Ackoff e Emery, um indivíduo ou sistema com propósito (1972/2017, pp. 34, 52–53). O que os separa é de onde vêm os fins. Para este artigo, adotamos portanto uma definição de trabalho própria, mais estreita do que um levantamento da literatura sustentaria. Uma inteligência geral, nesse sentido, seria um sistema com propósito, que seleciona seus objetivos além dos seus meios, cuja inteligência inclui o tipo criativo que Ackoff e Emery associam à consciência do ambiente e de si e consideram mais próximo da definição de Chein, relativa aos «propósitos imediatos e presumivelmente futuros do ator» ele mesmo (1972/2017, p. 52). Conjecturamos que tal capacidade emergiria no sistema, em vez de ser nele projetada. Por essa definição, que não é a que o benchmark adota, escolher objetivos é a condição que ainda separa os agentes aqui estudados de um agente geral. Não testamos consciência, e não fazemos afirmação alguma sobre consciência fenomênica ou senciência. O relato da fundação sobre os ambientes aponta na mesma direção, já que os descreve como tendo «mecânicas e objetivos determinísticos e fechados» e não lê a saturação do benchmark como «prova de ter alcançado a AGI». Ela também nomeia a inovação aberta entre as perguntas para a próxima geração de benchmarks (Kamradt, 2026b).

Se escolher objetivos está fora do que o benchmark pede, o passo que um agente pode dar dentro dele, e aquele para o qual o nosso foi construído, é a identificação do objetivo que cada jogo fixa, sem gastar as ações que a pontuação cobra. Esse passo une as duas perguntas abertas do segundo parágrafo, e abordamos ambas a partir da ponta oposta da literatura ali revisada, já que não aprendemos um modelo de mundo. Em vez disso, o agente mantém hipóteses concorrentes sobre o objetivo, nenhuma das quais pode proibir uma ação, e registra quais ações não mostraram efeito, de modo que as duas afirmações possam ser conferidas contra o jogo. Como a política padrão do agente é o modelo de linguagem, e não a nossa camada, a pergunta é se essas afirmações são corretas e se elas alcançam as escolhas do modelo. Tratamos portanto o componente que as faz como um instrumento de medida e variamos como ele está presente no agente, seja no modo jogando, no modo aconselhando ou no modo de último recurso (Seção 5). Concretamente, nosso agente é um explorador baseado em grafo acoplado ao *harness* de um agente LLM. O explorador roda busca em largura (BFS) até a fronteira de pares estado–ação não testados, na linha de Rudakov et al. (2025, p. 1). O *harness* foi primeiro um *fork* do *harness* Duck (Bessis et al., 2026a) e depois o notebook dfranzen (Franzen, 2026a), ambos descritos na Seção 3.1. Nossas contribuições são:

1. **Um certificado verificável de ausência de efeito** (Seção 3.2): uma afirmação registrada e revogável de que uma ação nada muda num estado, mascarada da lista do modelo nas configurações que podam (Seção 5). Registrar e revogar tais afirmações tem antecedentes, entre eles a guarda da base dfranzen (Seção 2).
2. **Detecção de estado oculto por falha de certificado ou morte inesperada** (Seção 3.2), com contadores por jogo (por gatilho nas versões posteriores).
3. **Um componente medido de identificação da condição de vitória** (H5, Seção 3.3): hipóteses de objetivo concorrentes cujo líder apenas ordena as ações candidatas, e cuja falha nomeada é um resultado de primeira classe. Com as famílias padrão, 50 de 66 vitórias locais não têm instância de hipótese verdadeira no quadro vencedor, e uma bancada de atribuição com oito braços não encontra efeito mensurável na pontuação de família alguma, de uma família ampla de proximidade, nem da ordenação aleatória.
4. **Um protocolo de verificação adversarial** (Seção 4; invariantes pré-registrados, rodadas independentes de *blue team* e *red team*, teste de mutação, artefatos congelados por hash) e a taxonomia de defeitos que ele produziu na linha Duck, onde a maioria dos defeitos ficava entre o que o agente afirma e o que o ambiente permite.

2 Trabalhos anteriores

Exploração em grafo sem modelo. Rudakov et al. (2025, pp. 1, 3–4) segmentam os quadros em componentes conexos de uma só cor, fazem o hash dos estados e então buscam a fronteira de pares estado–ação não testados. Seu explorador, sem treinamento, resolve uma mediana de 30 de 52 níveis da prévia e supera a melhor entrada baseada em LLM no *leaderboard* da prévia, cuja pontuação oficial relatam sem reexecutá-la. O custo do explorador cresce com o espaço de estados. Nosso explorador herda esse esqueleto, e o princípio é mais antigo. O Go-Explore lembra os estados visitados e primeiro retorna a um promissor, sem explorar, antes de explorar a partir dele. É uma busca em grafo sobre estados conflacionados cujas arestas precisam ser descobertas (Ecoffet et al., 2021, pp. 1, 3, 20). O validador do benchmark constrói um grafo assim, com nós baseados em hash fundidos entre trajetórias e ações inválidas registradas como transições que não mudam o estado. Seus nós também carregam o estado oculto do ambiente (ARC Prize Foundation, 2026, p. 9), ao passo que nossos estados de quadro mascarado não, e é isso que nossos gatilhos de estado oculto vigiam.

Modelos de mundo executáveis. Rodionov (2026, pp. 1, 4) faz um agente de programação manter um modelo de mundo em Python. O modelo é verificado contra as observações e refatorado rumo a abstrações mais simples, e o executor do plano para na primeira divergência. O OPINE-World (Courtis et al., 2026, pp. 2, 18, 27) acopla um protomodelo em linguagem natural, um estado de crença bayesiano sobre efeitos tipo–ação e um modelo em programa verificado por repetição. O estado de crença rastreia efeitos mistos como incerteza e testa refinamentos de ontologia, e remover a camada bayesiana custa 20,24 pontos de RHAE (78,40 → 58,16). O Twin (Skoutnev et al., 2026, pp. 1, 3, 22, 28) recusa qualquer ação pontuada até que seu gêmeo executável reproduza o registro *inteiro* da interação, e hipotetiza o objetivo antes de qualquer recompensa. A primeira hipótese de objetivo está correta em 87,2% dos níveis concluídos, mas a precisão de objetivo sobre todas as afirmações é 0,214, contra 79,4% de acurácia da dinâmica em linha (todas as previsões agregadas). O Tycho (Lehmann et al., 2026, pp. 1, 3, 24) formaliza os jogos como máquinas de Moore renderizadas e compara quatro políticas de manutenção de modelo numa execução no conjunto público por política,

sob orçamentos de inferência pareados. O reparo automático atinge a maior correspondência de transições (88,1%) e não pontua melhor, de modo que acurácia de transição e pontuação se separam nessa comparação.

Explorar, depois resolver. Liew (2026, pp. 1–3) condiciona um agente EXPLORE/VERIFY/PLAN a um substituto da entropia de crença. O estudo encontra cada um dos 25 jogos públicos alcançável por estratégias não inteligentes, de modo que o conjunto público «não consegue discriminar exploração inteligente de heurísticas triviais» (p. 1). A ressalva se aplica à nossa bancada local (Seção 5), construída sobre os mesmos jogos. Tsividis et al. (2021, pp. 3–5) aprendem condições de vitória e de derrota como parte de um modelo bayesiano em programa (EMPA), com curiosidade baseada em teoria gerando objetivos exploratórios. A Seção 3.3 os segue ao tratar o objetivo como um objeto a ser aprendido.

Conhecimento de no-op em *harnesses*. Num *fork* do Duck, o módulo `agentfix` de Le Grand (2026) tem chaves que medem o efeito de cada ação executada fora da faixa do HUD (exibição), anotam repetições de um no-op (estado, ação) provado (`repeat_of_known_no_op`) e recusam tais repetições. A recusa é limitada e deixa cada terceira tentativa reverificar. As três chaves estão desligadas nos padrões da versão publicada, e a recusa está desligada porque, segundo suas notas, as recusas «enganaram o modelo» numa versão anterior. Em outro *fork* do Duck, o laboratório arc-3 relata a detecção de ausência de impacto, a interrupção de uma ação exploratória cuja única mudança no tabuleiro é a faixa de HUD/lances do jogo, como a única vitória clara das suas ablações (sonpham-org, 2026). A média de duas passadas, excluído o jogo ft09, deu 1,62 contra 1,05 e 21 contra 15 níveis com orçamento de ações igual. O laboratório adverte que uma única execução não decide uma mudança de *harness*. Num terceiro *fork*, sobre a base dfranzen, um verificador de prefixo baseado em observação, testado à parte do modelo de linguagem nos jogos públicos, não fez melhor do que checar após cada ação, e foi entregue desligado (Shamim, 2026, pp. 1, 10). O explorador aprendido da mesma entrada, que penaliza ações com histórico de no-op, fez menos no-ops do que uma política aleatória (31,05 contra 47,69 por execução) e não pontuou mais (pp. 9, 11). Prever quais ações mudam o quadro também moveu a entrada vencedora da prévia de 2025, o StochasticGoose, uma rede convolucional treinada por aprendizado por reforço (ARC Prize Foundation, 2026, p. 20). A própria base dfranzen traz uma guarda, desligada por padrão (Franzen, 2026a). Ela registra no-ops (estado, ação) com borda mascarada por nível, recusa uma repetição a menos que o modelo a reemita de imediato, e descarta uma entrada que evidência posterior contradiga. O seu autor relata que guardas de no-op memorizado, entre outras, foram exploradas «sem benefício suficientemente claro para incluí-las nesta configuração» (Franzen, 2026b). Nosso certificado mantém esse registro e essa revogação, e acrescentamos e medimos a resposta a uma contradição, pela qual o nível é marcado como de estado oculto e seus outros certificados são anulados. A máscara da lista e os contadores por gatilho decorrem disso.

Indução de objetivo a partir de transições de nível. O agente Misfit (McCree, 2026) induz predicados de objetivo ordenados a partir de transições observadas (o suporte exige uma mudança de nível) em três famílias: `removed_all_of_class`, `agent_reached_class` e `count_of_class_equals_N`. Os predicados são pontuados por uma posterior com suavização de Laplace em que um nível que avança sem o predicado conta como contradição. A hipótese mais bem ordenada numa vitória é persistida como etiqueta, e nenhuma ação é tomada a partir dela. Em relação a essa implementação, a Seção 3.3 usa a hipótese líder para ordenar ações e mede o resultado.

Mesma base, outras alavancas. Trabalho concorrente sobre a mesma base dfranzen (Lam Dang, 2026) manteve o agente inalterado e trabalhou no motor de inferência. As mudanças foram poda de especialistas (*expert pruning*), concorrência, carregamento mais rápido e, depois, correções de

escalonador e de cache, e elevaram a pontuação no *leaderboard* de cerca de 26 para 33,68 e depois para 37,75 (pontuada depois do prazo). Toda mudança de *harness* que também submeteu pontuou abaixo do *harness* inalterado, inclusive uma guarda contra repetir ações no-op conhecidas (+3,9 ± 3,5 nos jogos públicos, −8,0 no *leaderboard*, uma tiragem no *leaderboard*), e a validação pública não acompanhou o *leaderboard* para mudanças no agente. Em seu relato, sob orçamento fixo de tempo de parede, o número de tokens que o modelo consegue gerar por jogo limita o que o agente pode fazer. Uma camada como a nossa consome esse mesmo orçamento. Outra equipe sobre a mesma base, com o *patch* declarado idêntico, chegou a 43,73 por serviço de inferência e por um escalonador que mantém 39 a 48 jogos ativos em vez de 10, e não relata ablação das suas mudanças de *harness* (Taguchi et al., 2026). Depois da competição, o participante classificado em sexto relatou a mesma divisão, em que a otimização do serviço de inferência foi «a MAIOR coisa a melhorar», e as mudanças de *harness* «morreram no ruído» (Zatravkin, 2026b). Os jogos públicos foram um preditor fraco mesmo para uma configuração de serviço, já que a execução recorde do Team Pandora nos jogos públicos (53,51) pontuou 26,90 no conjunto oculto, abaixo de uma configuração que pontuara menos nos jogos públicos (27,38; Team Pandora, 2026).

3 Método

3.1 O explorador (K) dentro do *harness*

O sistema acopla K ao *harness* de um agente LLM por um único gancho de análise, e duas bases foram usadas, ambas com esse gancho. K e seus certificados vêm de uma única linhagem de código, versionada e congelada por hash a cada submissão. A primeira é um *fork* do *harness* Duck (Bessis et al., 2026a), em que um modelo de linguagem local escreve e executa Python em chamadas de ferramenta curtas e repetidas. Ela produziu nossa primeira submissão pontuada (*leaderboard* 0,75, Seção 5), anterior ao H5, e foi então aposentada. Passamos à base dfranzen depois de ler os notebooks públicos da competição, quando a distância entre as nossas pontuações e as dos primeiros colocados mostrou que o explorador não pontuaria competitivamente nas condições da competição. Os notebooks públicos que pontuavam derivavam dessa base, e rodá-la sem modificação deu o controle pareado para os modos que se seguiram. A segunda é o notebook dfranzen do Marco 2 (Franzen, 2026a), ele mesmo um *fork* do *harness* Duck e publicado sob Apache-2.0 no repositório do seu autor. Nós o usamos como publicado, não por cópia de terceiros, e ele sustenta todas as versões posteriores. A linha Duck rodou o Qwen3.6-27B em FP8 (vrfai/Qwen3.6-27B-FP8, carregado do conjunto de dados Kaggle `driessmit1/vrfai-qwen3-6-27b-fp8-hf-snapshot`). A linha dfranzen roda o modelo da base, Intel-Qwen3.8-Flash-Next-W4A16-AutoRound (modelo Kaggle `dfranzen/intel-qwen3.8-flash-next-w4a16-autoround`), servido com um rascunhador de decodificação especulativa (`dfranzen/albucino-qwen3-8-flash-next-drafter`). Tomamos o modelo e o serviço publicados de cada base como referência e não otimizamos nenhum dos dois. Nas duas linhas, nossas mudanças no notebook da base são o gancho de análise, cujos ajustes (gatilhos e tetos) são a única adaptação a K, e, fora das reexecuções da competição, uma amostra fixa de três jogos e um teto de tempo para a fase A (Seção 5). As diferenças entre configurações sobre a mesma base vêm portanto da presença de K e não da computação, a condição que a Seção 8 propõe para uma trilha dedicada.

Substituímos a política de exploração por K, um explorador baseado em grafo em que os quadros recebem hash como estados com as células da faixa de exibição mascaradas. K mantém o grafo dirigido dos estados visitados e das ações testadas e planeja em largura até o estado mais próximo com ações não testadas. Ele também aprende um orçamento de ações por vida (as ações desde o

último reinício ou início de nível) quando mortes repetidas caem na mesma contagem. K carrega duas heurísticas de objetivo fixas herdadas da sua linhagem, mover um móvel rumo a um alvo (H1) e igualar uma região de trabalho a uma referência estática (H2), e elas apenas ordenam candidatos. H1 muda de alvo quando estagna num deles, e a orientação se desliga para o nível quando a discrepância de H2 para de cair ou o grafo do nível se esgota. Ambas foram escritas a partir de uma leitura dos 25 jogos públicos que também formam a bancada local, e H5 (Seção 3.3) substitui a orientação delas pela sua instância líder. Quando o grafo de um nível se esgota, K esquece todo resultado testado daquele nível, exceto as mortes, e testa de novo, no máximo oito vezes por nível, alternando com um reinício do nível.

Na primeira configuração (**modo jogando**), o modelo de linguagem propõe ações por padrão, e K observa e, onde configurado, poda a lista de ações (Seção 3.2). Em gatilhos nomeados (falha do modelo, estagnação, no-op repetido, pressão de tempo, com o rendimento medido numa variante), K assume o turno, e suas notas do grafo entram no prompt do modelo. Uma configuração posterior (**modo aconselhando**) desativou essas passagens de turno, de modo que nenhum gatilho dava a vez a K e uma falha do modelo voltava ao *harness* como na base sem modificação. A configuração final restringe K ainda mais (**modo de último recurso**, Seção 5), em que prompt e lista de ações são, por construção, idênticos byte a byte aos da base sem modificação até os 600 segundos finais do teto de cada jogo. Só então, com um nível ainda aberto, K pode jogar no máximo 150 ações por nível e 600 por jogo. O prompt nunca usa a palavra «certificado», e no modo de último recurso o modelo não vê texto algum do explorador. A versão v14.1 acrescenta a memória entre jogos da Seção 3.4.

O notebook submetido é reexecutado pelo Kaggle no conjunto oculto de jogos, com a internet desativada, como em toda sessão acelerada do Kaggle (ARC Prize Foundation, s.d.), e nosso notebook não recebe entrada humana. A execução oculta joga 110 jogos uma vez, numa única passada (Team Pandora, 2026; Zatravkin, 2026b). Desses, 55 são pontuados no *leaderboard* público e 55 no privado, segundo o relato de um participante (Zatravkin, 2026b).

3.2 Certificados de ausência de efeito e estado oculto (H8)

Um **certificado** é emitido para (estado, ação) quando uma única ação, fora de qualquer lote, é executada num nível ainda não marcado como de estado oculto, o jogo não termina, e o quadro sucessor difere do quadro atual no máximo dentro da faixa de exibição mascarada. As chaves certificadas são removidas da lista de ações válidas dada ao modelo de linguagem (mascaramento de ação por no-op observado, *action masking*), *a menos que* a remoção esvaziasse a lista. Nesse caso, a lista é mantida inteira, e o texto de aconselhamento o diz (invariante I10, Seção 4). Um certificado é **falseado** quando uma execução posterior o contradiz, e o falseamento **revoga** o certificado e anula os outros certificados daquele nível. Os certificados de um nível também são revogados quando K reabre suas ações testadas (Seção 3.1), de modo que o que a máscara havia descartado volta à lista do modelo, mas as mortes nunca são reabertas. Todos os certificados são revogados quando K reaprende a máscara de exibição e reindexa seus estados.

Um nível é detectado como de **estado oculto** por qualquer de dois gatilhos nomeados. O gatilho (i) é o falseamento como acima, incluindo uma morte depois de um no-op mesmo sem certificado prévio. O gatilho (ii) é uma morte numa ação que K já considerava segura naquele estado (sem efeito, uma mudança ou progresso) ou no orçamento de vida aprendido (`morte_oculta`). Num nível de estado oculto, toda declaração posterior de no-op no prompt do modelo é rebaixada de «provado que NADA muda» para «não mudou nada aqui antes, mas este nível tem estado oculto, logo isso NÃO é uma prova». Resultados de no-op herdados através de uma fusão de estados são rebaixados do mesmo modo até serem reobservados. A tabela-resumo da execução conta, por jogo, as emissões,

os falseamentos, as revogações na reindexação e na reabertura, e as chaves mascaradas, e lista os níveis de estado oculto (por gatilho a partir da v11). A Tabela 1 os relata para a v4 (não submetida) e para a v5 (a submissão pontuada), na fase A da linha Duck (Seção 5). A Figura 2 desenha o percurso do certificado e onde K age em cada modo.

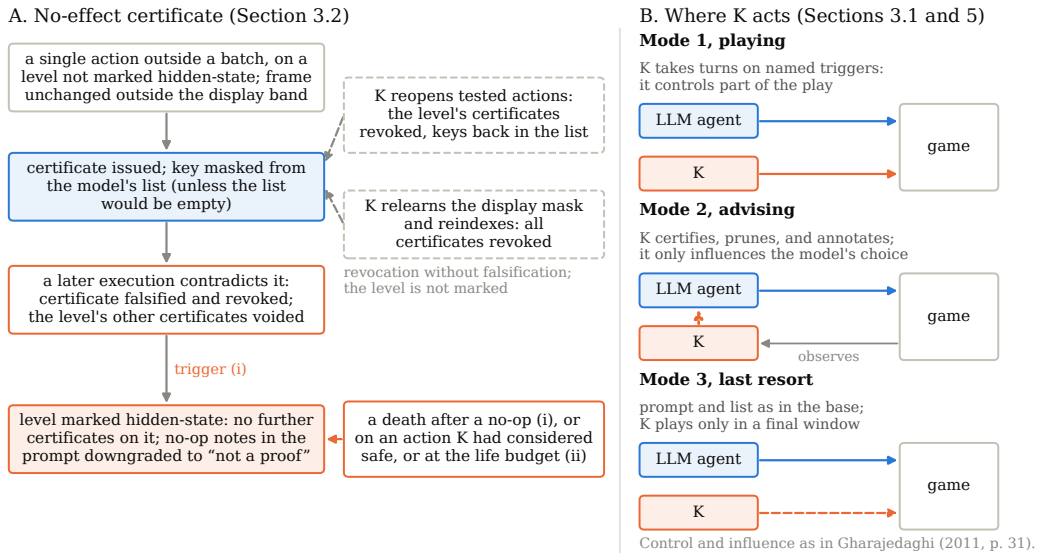


Figura 2: A, o certificado de ausência de efeito e a marca de estado oculto (esta seção); nenhum certificado é emitido num nível depois de marcado. B, onde K age em cada modo de presença (Seções 3.1 e 5). Nos termos de Gharajedaghi, controlar é agir de modo necessário e suficiente para um resultado, e influenciar é agir apenas como seu coprodutor (2011, p. 31). K controla parte do jogo nos modos 1 e 3 e apenas influencia o modelo no modo 2.

Um certificado se apoia no que Bateson (1972) chama de pressuposto do contexto repetível, «uma premissa necessária para qualquer teoria que defina "aprendizagem" como mudança» (p. 292). Sem ele, argumenta, «tudo seria aprendizagem zero» (p. 288), uma especificidade de resposta que «não está sujeita a correção» (p. 293). Para Bateson a premissa é do teórico, mas ela «não é um mero instrumento da nossa descrição» e contém a hipótese de que a experiência do organismo é pontuada em contextos «que podem ser igualados ou diferenciados pelo organismo» (p. 292). Nosso agente transforma essa hipótese num teste ao supor que o mesmo hash mascarado, o correspondente do que Bateson chama de marcador de contexto (p. 290), marca o mesmo contexto. Marcar um nível como de estado oculto retira esse pressuposto para o nível uma vez que seu marcador se mostre conflacionar dois contextos, como no relato de Bateson sobre a neurose experimental, em que os marcadores de contexto «se tornam enganosos» (p. 297).

3.3 Identificação da condição de vitória (H5)

O componente se apoia na premissa deste trabalho, a de que um agente deve chegar a um jogo com técnicas abstratas para descobrir o que se pede, em vez de explorar exaustivamente e abstrair depois. A Seção 5 mede a linhagem de exploração-primeiro contra a qual essa premissa argumenta. Sobre K, acrescentamos um componente de indução de objetivo (H5), fechado no laço de ação de K, no espírito das condições de término aprendidas do EMPA e das famílias de predicados do Misfit. Nos turnos do modelo, H5 apenas observa. Cinco famílias de predicados são definidas: móvel-sobre-alvo

Tabela 1: Contadores de certificados por jogo, fase A da linha Duck (três jogos públicos, teto de 7.920 s por jogo, executados concorrentemente; uma tiragem). «Modelo em certificado» conta ações do modelo sobre uma chave certificada; todas as revogações aqui são na reindexação (nenhuma na reabertura); as chaves mascaradas são somadas sobre os prompts. Os níveis de estado oculto são listados por índice; o contador por gatilho não era impresso nessas execuções, logo o gatilho é inferido (sem certificado e sem falseamento ⇒ gatilho (ii)). A v4 é anterior à correção que fez mortes em arestas de mudança ou progresso, ou no orçamento de vida, marcarem o nível (Tabela 2, v5), logo sua coluna de estado oculto não é comparável com a da v5.

versão	jogo	níveis	modelo em certificado	certificados	falseados	revogados	chaves mascaradas	níveis de estado oculto
v4	ls20	1	25	142	0	119	167	—
v4	tu93	3	0	8	0	3	20	—
v4	vc33	2	27	233	2	5	0	1
v5	ls20	1	35	50	1	43	217	1
v5	tu93	3	0	0	0	0	0	3 (gatilho (ii))
v5	vc33	1	20	84	5	0	0	0, 1

(O1), remoção de classe (O3), região-igual-a-referência (G1), objetos dentro de um contorno fechado (G2) e simetria especular (G4). As instâncias são criadas a partir de quadros observados assim que sua evidência aparece, de modo que a remoção de classe, por exemplo, é criada só depois que uma classe é vista desaparecer, e a simetria especular só no início de um nível. As instâncias são avaliadas nos estados conhecidos do grafo de K, e cada uma carrega um escalar de progresso que deve decrescer (para O3, as células restantes da classe).

Tolman (1948, pp. 201–202) credita a Krech a noção de «hipóteses» em ratos. São sequências sistemáticas de escolhas, como todas as portas da direita e depois todas as portas escuras, que Tolman lê como mapas cognitivos montados «de modo tentativo, para serem experimentados primeiro um e depois outro até que, se possível, se encontre um que funcione» (p. 202). Nossas hipóteses também são experimentadas em sucessão, embora ordenadas por evidência. Ao contrário das de Krech, que Tolman toma como mostra do «caráter autoiniciado» da aprendizagem (p. 203), as nossas não são do próprio agente, já que as famílias são pressupostos do projetista. Churchman (1971) observa que todas as máquinas de inteligência artificial «pressupõem implicitamente uma capacidade que se supõe que aprendam, desde que incluamos os pressupostos que seus projetistas usaram para construí-las» (p. 131). Nosso agente, do mesmo modo, revisa hipóteses apenas dentro de um espaço que nós autoramos.

A hipótese líder **apenas ordena** candidatos, selecionando dentro da camada atual de candidatos não tentados e reordenando estados de fronteira a até três passos do mais próximo, e nunca exclui uma ação. Um líder é substituído quando estagna (40 ações sem melhorar seu melhor progresso), quando o grafo do nível se esgota, ou quando é **eliminado**. Uma hipótese é eliminada quando seu predicado exato se torna verdadeiro num quadro estável, sem mudança de nível, depois que todo tipo legal de ação naquele estado foi tentado. O objetivo que ela nomeava foi então alcançado, e o jogo não terminou. Numa vitória, as instâncias são conferidas no quadro vencedor, e a família do líder (se o líder é verdadeiro) ou da instância verdadeira mais bem ordenada recebe evidência extra no nível seguinte. Só um líder verdadeiro conta como identificação (Seção 5). As versões submetidas desativam O1 por padrão, depois que o *red team*, sob o protocolo da Seção 4, mediu um custo de pontuação sobre oito sementes e uma latência que cresce quadraticamente em grades de 64×64. A bancada da Seção 5 o reativa como um braço.

3.4 Memória entre jogos (submetida, v14.1)

A versão v14.1 deixa H5 carregar algo entre jogos, sob uma regra: transferir a ideia, não o lance. Um armazém de todo o processo, com trava porque os jogos rodam em *threads* paralelas, mantém duas contagens só a partir de vitórias não ambíguas (exatamente uma instância verdadeira no quadro vencedor): vitórias por família de predicado e vitórias por esquema abstrato, uma família junto com os papéis dos seus argumentos. Um papel não tem cor nem coordenada. Cada cor se torna sua posição de raridade entre as cores não de fundo do primeiro quadro do nível (limitada a 3) mais um sinalizador de se ela se move sob ação, de modo que o papel é invariante a permutação de cores, a menos de empates na área da cor. A evidência dentro do jogo sempre decide. O esquema só desempata para uma instância já acoplada a ação (seu progresso caiu sob uma ação), e a contagem por família desempata depois disso. O braço conta como exercitado num jogo só se uma decisão de K teve seu líder mudado pelo armazém (`k_cross_decisao` > 0), critério fixado antes da execução.

O desenho segue um resultado antigo. Harlow (1949, pp. 52–56) deu a oito macacos 344 problemas de discriminação em sequência, e em cada um o macaco tinha de descobrir qual de dois objetos novos cobria o alimento. Nos primeiros problemas, uma tentativa com um par novo deixava os macacos pouco acima do acaso na seguinte. Nos últimos, essa única tentativa bastava para escolher corretamente quase sempre (p. 56). Ele chamou o que haviam adquirido de conjunto de aprendizagem (*learning set*), «aprender a aprender um tipo de problema» (p. 53). O que se transferia não eram as respostas, pois ele duvidava que os macacos pudessem escolher muito acima do acaso na primeira tentativa de um problema que já haviam resolvido (p. 59). Bateson lê o mesmo padrão nos experimentos de Hull sobre aprendizagem mecânica, em que os sujeitos melhoravam de sessão para sessão. Ele atribui o ganho a «algum tipo de adaptação ao contexto», já que lembrar as sílabas específicas «seria mais provavelmente um estorvo» (1972, p. 295). Ele classifica os conjuntos de aprendizagem de Harlow como «um caso bastante especial» da mesma categoria, a Aprendizagem II, uma mudança no conjunto de alternativas de que o aprendiz escolhe (pp. 293, 295–296). Nossa memória visa a esse tipo de transferência, guardando que tipo de objetivo venceu, nunca os lances que o venceram. Ackoff e Emery descrevem uma aprendizagem mais estreita em seu sistema buscador de objetivo com memória, um rato eletrônico resolvedor de labirintos, um autômato que pode ser programado para tomar um caminho de solução em tentativas posteriores num labirinto conhecido (1972/2017, pp. 30–31). A memória entre jogos visa ao caso mais difícil, um jogo novo com um objetivo diferente, o equivalente de um labirinto desconhecido. Como não foi exercitada na fase A (Seção 5), não reivindicamos aprendizagem em nenhum dos dois sentidos.

4 Protocolo de verificação adversarial

O agente foi desenvolvido sob um protocolo fixado num pré-registro escrito antes da primeira versão (27 de outubro) e aplicado a toda versão da linha Duck. Ele fixou invariantes (I1–I10): nenhuma exceção da nossa camada alcançando o *harness*, estado por jogo, ações legais, limites de tempo, uma reexecução intocada, sem rede, fontes embutidas iguais aos seus hashes, e uma máscara que nunca esvazia a lista de ações. Fixou também afirmações (C1–C4): que a fase A roda sem exceção e fecha um nível, que a reexecução pontua acima de zero, que uma previsão de pontuação é lacrada antes da execução, e que os contadores são não nulos. Um *blue team* implementou cada versão e sua suíte de testes, e um *red team* independente a atacou. O *red team* era sempre um modelo diferente do *blue team*, numa sessão nova, com o artefato congelado e seu manifesto de hashes como alvo. Ele entregava um veredito, LIBERAR, LIBERAR COM RESSALVAS ou NÃO LIBERAR, com uma prova reproduzível por achado. As correções voltavam com testes de regressão e com mutantes,

reintroduções deliberadas de cada defeito que têm de falhar nos testes novos. Testes que passam se assemelham ao que Churchman, seguindo Singer, chama de leituras, e «se as leituras replicadas concordam, não podemos inferir que o sistema esteja funcionando corretamente» (Churchman, 1971, p. 191). Sua regra é refinar a investigação «até que o sistema alcance um nível de refinamento das suas leituras em que nem todas as leituras concordem» (p. 193). Os mutantes são nossa conferência de que os testes conseguem discordar, e sua numeração chegou a M85 na v14.1, cujos oito mutantes novos foram todos mortos na sua rodada de *red team*. Nenhuma versão com um achado bloqueante aberto foi jamais submetida, e a rodada de *red team* da v7 foi ela mesma superada pela da v8.

Entre as versões da linha Duck (v1–v7), a maioria dos defeitos que os *red teams* encontraram compartilha um caráter (Tabela 2). São lugares em que o que o agente *afirma*, no seu prompt ou no seu plano, diverge do que o ambiente *permite*, e não casos de a busca simplesmente falhar. O restante é contabilidade através de fronteiras (Tabela 2) e uma aresta deixada sem aprender. Oferecemos a taxonomia como lista de conferência para trabalho em *harness*: legalidade nas revisitas, morte como falseamento, fronteiras de lote, proveniência de estados fundidos e honestidade do aconselhamento.

Tabela 2: Defeitos encontrados pelos *red teams* na linha Duck, listados sob a versão que os corrigiu (cada um foi encontrado na versão anterior e corrigido com um teste de regressão e um mutante).

versão	**principais defeitos encontrados**
v2	contador de vidas herdado através de uma fronteira de nível (reinício espúrio)
v3	clique escrito num enum compartilhado; certificado sobrevivendo à morte do agente; plano cruzando uma fronteira de nível num lote
v4	ação fora da lista legal num estado revisitado; certificado reemitido no quadro de fim de jogo depois de uma reabertura ou reindexação
v5	lista de ações inventada quando só RESET era legal; aviso de máscara alegando remoções que não haviam ocorrido; dica de «no-op provado» num nível falseado; morte oculta numa aresta de mudança ou progresso, ou no orçamento de vida, sem marcar o nível
v6	aresta não aprendida para o estado só-RESET (laço); BFS sobre uma aresta ilegal herdada de uma fusão; dica de «provado» para um no-op herdado de outro quadro
v7	dica de «ainda não tentado» fora da lista legal; estado inicial órfão depois da reindexação; BFS através de um estado fundido de duas telas sob uma ação legal em só uma delas

5 Experimentos

Placar (oficial). As pontuações estão em pontos percentuais (0–100), como no *leaderboard*. Nossa primeira submissão pontuada (*fork* do Duck + K, versão v5 da linha) concluiu com **0,75** (ref. 57852519 do *leaderboard*). Nossa medida da outra base é o notebook dfranzen intacto, que concluiu com **26,74** (ref. 57881859), dentro da dispersão de 25,8 ± 3,9 das cópias inalteradas submetidas por outras equipes, conforme relata Lam Dang (2026). A versão v14.1 (modo de último recurso mais a memória entre jogos) foi submetida em 2 de novembro às 22:30 UTC. Sua reexecução no conjunto oculto, limitada a 9 horas, foi concluída depois do prazo da competição, logo não é uma entrada elegível e não aparece no *leaderboard*. Sua pontuação, **28,41** (ref. 57923098), é relatada aqui como uma medição de uma tiragem, 1,67 ponto acima da nossa tiragem da base intacta e dentro da

dispersão de *leaderboard* relatada para a base. Nenhum efeito é inferido, e, como a memória não foi exercitada na fase A, a pontuação não pode ser atribuída a ela. Duas das nossas entradas foram portanto elegíveis, das quais a base intacta (26,74) é a pontuação da equipe no *leaderboard*, e a primeira, 0,75, é a única entrada elegível que carrega nosso agente.

Fase A (jogos públicos). Cada versão candidata rodou primeiro os três jogos públicos dentro do kernel Kaggle. A Tabela 1 dá os contadores de certificados e os níveis de estado oculto da v4 e da v5, incluindo um nível marcado como oculto pelo gatilho (ii) com zero certificados (inferido). Houve uma tiragem por versão, e a dispersão da fase A da linha não foi medida. Na ablação abaixo, braços com três rodadas válidas variam de 2,6 a 11,7 pontos (DP amostral) de rodada para rodada. Uma única diferença de fase A entre duas versões não é portanto evidência sobre o explorador ou seus certificados.

Três modos de presença (a comparação central). Sob o `(h/a)^2` por nível da RHAE, toda ação conta, inclusive as do explorador. (Modo 1, *jogando.*) Na configuração híbrida, K tomou a maioria dos turnos, e na fase A da v5 um jogo teve 210 turnos do modelo contra 370 de K e gastou 857 ações, 770 delas num nível que não fechou. A submissão pontuada ficou em 0,75, entre K puro nos jogos públicos (≈0,4 na bancada local descrita abaixo, um conjunto de jogos diferente) e a base dfranzen intacta no *leaderboard* (26,74). As duas bases diferem, e nenhum controle pareado da base Duck foi pontuado, logo isto é evidência compatível com o custo das ações do explorador, não uma estimativa do seu efeito. Separadamente, na mesma base dfranzen e nos mesmos jogos públicos, duas rodadas de fase A com K jogando (código v9.1) pontuaram **17,08** e **38,16** (média 27,62), contra 72,32–77,43 da base. Essas duas rodadas são descritivas (n = 2, sob a regra das três rodadas) e não são um pareamento das pontuações do *leaderboard*.

(Modo 2, *aconselhando.*) K então parou de jogar e, em vez disso, observou cada transição do modelo, emitiu certificados, podou chaves certificadas da lista do modelo e anotou o prompt. Na sua primeira rodada de fase A (três jogos públicos, 2.700 s cada), o conselheiro emitiu 28 certificados e fez 57 remoções de chave (somadas sobre os turnos). Registrou zero falseamentos, o que aqui não é evidência, já que o modelo nunca executou um par (estado, ação) certificado (model_act_certified = 0), e um certificado só é falseado ao ser executado. A ablação compara o conselheiro com a base sem modificação sob condições pareadas (mesmos jogos, tetos e concorrência, com a inicialização do servidor não controlada). Só rodadas que passaram nas conferências de validade são lidas, e uma rodada da base, perdida porque o servidor do modelo nunca ficou pronto, foi reexecutada. A base pontuou **77,43, 74,98, 72,32** (média 74,91, DP amostral 2,56) e o conselheiro **68,85, 59,42, 76,86** (68,37, 8,73). Fixamos a regra de leitura depois de uma primeira rodada-piloto da base e do conselheiro (77,43 e 68,85) e antes de coletar as rodadas seguintes, e as médias relatadas mantêm esse piloto. A regra pede três rodadas por braço e lê um efeito só se a diferença das médias excede duas vezes o DP dentro do braço. Depois das rodadas, lemos esse DP de modo conservador como o maior DP amostral do par, e por essa regra o conselheiro não mostra efeito mensurável (6,5 pontos contra uma barra de 17,5). O braço de texto de aconselhamento sem poda tem três rodadas válidas (**42,09, 53,55, 65,57**, com média 53,74 e DP 11,74), depois que uma rodada adicional, a segunda, foi excluída por uma exceção na nossa camada. O braço fica abaixo de toda rodada da base, mas também sob a barra (21,2 contra 23,5), logo não mostra efeito mensurável, o que não é o mesmo que ausência de custo. Nas nove rodadas válidas, a dispersão é grande em todo jogo (ls20 8,46–53,57, tu93 46,67–100, vc33 28,35–100), e esta é a evidência em condições pareadas do artigo (Figura 3).

Diagnosticamos a maior lacuna entre as duas rodadas-piloto, num jogo (ls20). O raciocínio visível do modelo nunca mencionou as notas de aconselhamento, e suas duas oscilações caras, estratégias que o próprio modelo escreveu, chamavam ações diretamente num laço de código que contornava a

lista podada. Isso não exclui um efeito das notas, ou da poda que o prompt não nomeava, em outro lugar da execução, mas não conseguimos atribuir a lacuna à camada de aconselhamento a partir desse rastro.

(Modo 3, *último recurso*.) A configuração final (Seção 3.1) mantém prompt e lista de ações idênticos aos da base e deixa K jogar só nos 600 segundos finais do teto de cada jogo. Nada nessa construção torna o modo aditivo, já que a janela substitui os turnos do modelo, e o modelo vê as ações do explorador no seu histórico. A construção limita a intervenção em tempo e em ações. A perda de pontuação não é limitada e não foi medida. A janela de 600 segundos era 22% de um jogo de fase A de 2.700 segundos e é cerca de 2% do teto do conjunto oculto, logo as execuções de fase A desse modo (72,62 sobre três jogos na v14.1) não são comparáveis com a base. A janela comparável (a mesma fração de 1,88%, 50,75 s) foi fixada depois dessas execuções, e sua tiragem no conjunto oculto é o 28,41 relatado acima, contra 26,74 da base intacta.

Segundo lote (7 de novembro). Depois da competição, rodamos os quatro notebooks públicos da Seção 10 como um segundo lote sobre a mesma base, jogos e tetos (2.700 s por jogo, três jogos concorrentemente), com o braço de controle executado no mesmo dia que os outros braços. O controle pontuou **41,28, 41,35, 52,38, 74,18** (média 52,30, DP amostral 15,49). A quarta rodada do controle foi um reenvio que levava o texto revisto dos notebooks, e declaramos antes do seu resultado que toda rodada válida do controle seria lida, logo a base tem quatro rodadas e cada modo, três. A dispersão dentro de um dia é tão grande por jogo quanto no primeiro lote: nas quatro rodadas do controle, vc33 variou de 21,43 a 100,00, tu93 de 32,08 a 100,00 e ls20 de 16,97 a 35,71, e as médias das rodadas de 41,28 a 74,18. Sob a regra de leitura acima, um braço registra um efeito neste lote só se sua média difere da do controle por mais de duas vezes o maior DP amostral do par, que aqui é de 45 a 48 pontos, já que o DP de todo braço excedeu 15. K jogando pontuou **21,03, 64,06, 61,35** (média 48,81, DP 24,10) e não mostra efeito mensurável. Na sua primeira rodada, K tomou 833 de 858 turnos em ls20 pelo gatilho de pressão de tempo, e esse jogo pontuou 0,00. Na segunda, tomou 346 de 414 turnos em ls20, e a rodada pontuou 64,06. Na terceira, tomou 297 de 381, e a rodada pontuou 61,35 (Tabela 3). As duas primeiras rodadas estão 43 pontos distantes, razão pela qual nenhuma rodada isolada de fase A é lida como evidência. K aconselhando pontuou **50,80, 29,06, 74,04** (média 51,30, DP 22,49), sem turno algum tomado por K, e não mostra efeito mensurável. O braço de último recurso rodou com a janela comparável de 50,75 s (1,88% do teto de 2.700 s, a fração da reexecução no conjunto oculto) e pontuou **39,51, 81,64, 39,78** (média 53,64, DP 24,25). K executou 79, 45 e 63 ações dentro da janela (somadas sobre os três jogos), e o braço não mostra efeito mensurável. Uma rodada adicional de último recurso (65,26) foi invalidada por nove exceções da nossa camada em vc33 e reexecutada, e sua substituta é a terceira rodada lida. Esta é a primeira medição de fase A desse modo sob uma janela comparável com a base. Com três ou quatro rodadas por braço e DPs acima de 15, nenhum efeito menor do que cerca de 45 pontos teria sido lido neste lote.

Memória entre jogos (v14.1, fase A). O armazém recebeu dois registros (família G2, de ls20) e mudou o líder 82 vezes em ls20, contagem que inclui turnos do modelo. Nenhuma decisão de K foi mudada (`k_cross_decisao` = 0 nos três jogos), logo, pela regra pré-fixada, o braço não foi exercitado e não tiramos conclusão. Em vc33, o modelo fechou todos os sete níveis antes de a janela de último recurso abrir.

Lições de método da fase A. As comparações de fase A são frágeis de duas maneiras que não controlamos de início. Primeiro, com seis jogos numa GPU, o modelo tomou de 40 a 74 turnos por jogo, contra 49 a 101 com três jogos, em execuções que também diferiam em janela e configuração. Isto é portanto uma observação e não uma lentidão medida. Segundo, a etapa de carregamento de

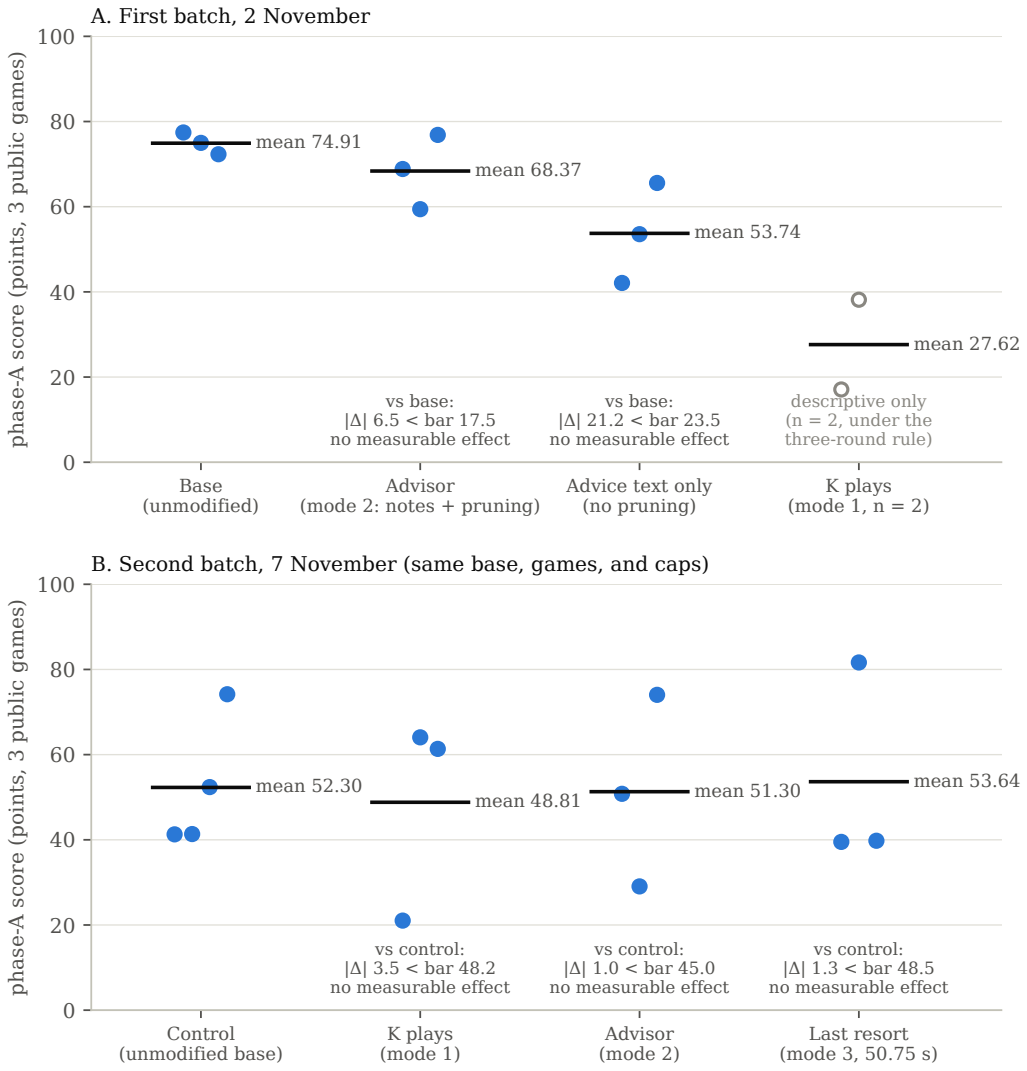


Figura 3: Rodadas de fase A por braço na mesma base dfranzen, jogos e tetos (três jogos públicos; um ponto por rodada válida; traços pretos são médias; pontos vazados marcam um braço com menos de três rodadas, só descritivo). A, primeiro lote, 2 de novembro. B, segundo lote, 7 de novembro, com o controle executado no mesmo dia que os braços. A regra, fixada depois de uma primeira rodada-piloto da base e do conselheiro e antes das rodadas seguintes, lê um efeito só se a diferença para a base excede duas vezes o DP dentro do braço, lido de modo conservador como o maior DP amostral do par, e nenhum braço a passa em nenhum dos lotes. Em A, o braço K jogando tem duas rodadas e não é um pareamento das pontuações do *leaderboard*. Em B, todo braço tem três rodadas e o controle quatro, e a barra é de cerca de 45 pontos, já que o DP de todo braço excedeu 15. Os rótulos da figura estão em inglês, como no original.

Tabela 3: Segundo lote, fase A na base dfranzen (três jogos públicos, teto de 2.700 s por jogo, executados concorrentemente; RHAE em pontos percentuais; ações somadas sobre os três jogos). Um braço é lido quando tem três rodadas válidas. Validade por rodada: hashes dos módulos iguais ao manifesto congelado, nenhum traceback, nenhuma exceção da nossa camada, e contadores de K não nulos nos braços com K. A execução do *fork* do Duck do mesmo dia não está nesta tabela, já que as bases diferem.

braço	rodada	média	ls20	tu93	vc33	ações	nota
controle (base sem modificação)	1	41,28	22,41	80,00	21,43	752	
controle	2	41,35	16,97	32,08	75,00	871	
controle	3	52,38	35,71	100,00	21,43	831	
controle	4	74,18	32,55	89,99	100,00	1.037	reenvio só de texto; lida (declarado antes do resultado)
K jogando	1	21,03	0,00	9,52	53,57	1.734	K tomou 833 de 858 turnos em ls20
K jogando	2	64,06	12,18	80,00	100,00	1.516	K tomou 346 de 414 turnos em ls20
K jogando	3	61,35	30,18	53,89	100,00	1.531	K tomou 297 de 381 turnos em ls20
K aconselhando	1	50,80	28,91	78,63	44,87	913	nenhum turno de K
K aconselhando	2	29,06	32,95	33,16	21,08	785	nenhum turno de K
K aconselhando	3	74,04	22,12	100,00	100,00	1.068	nenhum turno de K
último recurso (janela de 50,75 s)	1	39,51	34,88	62,22	21,43	920	79 ações de K na janela
último recurso	2 (inválida)	65,26	30,80	64,99	100,00	1.045	nove exceções da nossa camada em vc33; não lida; reexecutada como rodada 4
último recurso	3	81,64	44,92	100,00	100,00	976	45 ações de K na janela
último recurso	4 (reexecução da 2)	39,78	35,71	55,64	27,99	988	63 ações de K na janela

pesos do servidor do modelo levou de 305 a 1.494 segundos em oito execuções de fase A (em quatro delas os jogos foram liberados enquanto o servidor ainda carregava). Isso é consistente com carregar um *checkpoint* de ~190 GB de um sistema de arquivos em rede, o que copiar os fragmentos para a RAM reduz a cerca de 300 segundos numa sessão interativa (Lam Dang, 2026). Numa rodada medida da base sem modificação, a inferência do modelo ocupou 81% do tempo de jogo, e não perfilamos os ≈19% restantes, que incluem uma *sandbox* nova por chamada e todo o histórico serializado por ação. Relatamos deltas de fase A só sob concorrência e tetos pareados. Como só uma submissão pontuada pode ser selecionada, uma última submissão precisa do orçamento completo de reexecução (aqui 9 horas), mais margem para fila e preparação, antes do prazo de seleção. A nossa não o teve, e outra equipe na mesma base relata ter perdido a seleção do mesmo modo (Lam Dang, 2026). Uma terceira fragilidade apareceu entre lotes. O braço de controle do segundo lote, executado em 7 de novembro com o mesmo código de fase A do primeiro (árvore sintática abstrata igual à do notebook de amostra intacta), pontuou 52,30 em média sobre quatro rodadas, contra 74,91 em 2 de novembro, cerca de 23 pontos abaixo, com um DP dentro do dia (15,49) seis vezes o do primeiro lote (2,56). Os dois lotes diferem em pelo menos uma condição medida, já que em 7 de novembro o servidor do modelo ficou pronto um a dois minutos depois do início dos jogos, ao passo que em 2 de novembro o carregamento levou de 305 a 1.494 segundos. Não medimos a causa, não agregamos os lotes, e toda comparação neste artigo é lida dentro do lote em que foi executada.

Exploração-primeiro, medida sozinha. Sem modelo de linguagem algum, uma bancada local (25 jogos públicos, 1.500 passos, 4 sementes, pontuada pela RHAE do toolkit nos pontos percentuais do *leaderboard*) mediu a linhagem sobre a qual K é construído. Essa linhagem começa com exploração em grafo e depois acrescenta as duas heurísticas de objetivo fixas da Seção 3.1. As execuções estão registradas de 29 de setembro a 3 de novembro. O código das execuções anteriores a 26 de outubro não está versionado, e só o K atual sem H5 (K0, Tabela 4) foi reexecutado. Substituir um agente de memória operacional por um grafo de estados e BFS até a fronteira não testada elevou os níveis concluídos de 9,5 para 14,25 de 183 por semente (pareado, mesmas sementes). Numa ablação do mesmo dia, nas mesmas sementes, remover a BFS do seu sucessor reduziu os níveis concluídos a menos da metade (15,5 → 6,75), mas a pontuação mal se moveu (0,273 contra 0,260). Nos níveis que conclui, a linhagem toma uma mediana de cerca de 13 vezes as ações da linha de base humana, tanto antes das heurísticas de objetivo quanto depois delas. Só 3–8% das suas conclusões ficam na contagem humana ou abaixo, e, como a pontuação por nível eleva ao quadrado a razão de ações,

as outras valem quase nada. Ela estabiliza em cerca de 16 de 183 níveis e uma pontuação perto de 0,39 em dois conjuntos de sementes. K não tem nenhuma das máquinas da busca por novidade, o método de Lehman e Stanley que recompensa divergir de comportamentos anteriores em vez de progredir rumo ao objetivo, e não tem arquivo de comportamentos passados nem medida de esparsidade. O custo aqui medido pertence portanto ao nosso explorador, e não ao método de Lehman e Stanley. Eles respondem à objeção de que a busca por novidade é busca exaustiva notando que o espaço de comportamentos pode com frequência colapsar num número manejável de pontos (2011, pp. 198–199). Listam entre seus limites o de não ter viés para otimização uma vez encontrada uma solução (p. 213), e uma pontuação por ação cobra esse limite. Sugerem que pode ser mais eficiente buscar novidade primeiro e ajustar pelo objetivo depois (p. 213), o inverso da premissa de objetivo-primeiro da Seção 3.3. A bancada mede o explorador sozinho nos jogos públicos e não a camada dentro de uma base de modelo de linguagem, logo é consistente com o modo 1 e não identifica o seu efeito.

Bancada local (atribuição de H5). Oito braços rodaram na mesma bancada, K sozinho, sobre o arquivo de K submetido, e cada braço liga ou desliga famílias ou a ordenação pelos sinalizadores da bancada. Os braços são lidos por uma regra registrada enquanto rodavam, depois de o braço K0 ter terminado. Sob ela, um efeito exige uma diferença de médias pareada além de duas vezes o DP entre sementes do braço padrão. Tomamos depois o DP populacional sobre quatro sementes (0,0817, logo a barra é 0,1634), e com o DP amostral a barra seria 0,1886. Nenhum braço passa qualquer das duas (Tabela 4), e a maior diferença é 0,0515. As contagens de identificação são o resultado que enfatizamos (Figura 4). Com as famílias padrão, 50 de 66 vitórias não têm instância de hipótese verdadeira no quadro vencedor (49 de 66 na versão medida primeiro), e o líder antes da vitória era verdadeiro em 2. Chamamos essas vitórias de anomalias. O quadro vencedor é o último quadro observado antes da mudança de nível, e as instâncias conferidas são as ainda não eliminadas (incluídas as expiradas). «Verdadeiro» aqui significa que um predicado que implementamos vale no quadro, não que ele é o objetivo do jogo, e uma vitória é ambígua quando mais de uma instância é verdadeira. Remover G2 coincidiu com uma queda nas vitórias com instância verdadeira de 16 de 66 para 5 de 64 nesta bancada, em que as trajetórias também mudaram, logo isto é uma diferença de cobertura e não uma decomposição pareada. Reativar O1 reduz as anomalias a 19 de 63, mas marca 327 instâncias verdadeiras em 63 vitórias, 43 delas ambíguas, sem ganho de pontuação, logo mede a amplitude de um predicado de proximidade e não a identificação. Uma prioridade aleatória fixa por instância deixa 48 de 63 anomalias, logo a ordenação não explica a maioria das correspondências ausentes, embora possa importar em jogos particulares. Numa bancada pareada de 8 sementes, gerações anteriores de H5 mediram $-0{,}062 \pm 0{,}025$ (v9) e $-0{,}006 \pm 0{,}028$ (v9.1, O1 desligado) contra K puro (diferença pareada média ± erro-padrão sobre as oito sementes).

6 Transferência além do ARC-AGI-3

Três partes não são específicas dos jogos deste benchmark. Um certificado de ausência de efeito precisa apenas de um conjunto discreto de ações e de um sucessor observável, junto com um modo estável de dizer «o mesmo estado» (aqui, um hash mascarado), e falseamento e revogação são contabilidade. A detecção de estado oculto por falha de certificado é, em forma geral, um teste de *perceptual aliasing* (duas situações compartilhando um estado observado) que qualquer agente interativo pode rodar sobre seu registro de ações. O protocolo adversarial da Seção 4 se aplica literalmente a qualquer artefato de agente versionado. Os três modos se transferem só como hipótese de desenho. Onde as ações de um explorador auxiliar entram na pontuação do agente principal, deixá-lo jogar deve ser custoso sob pontuação por ação, já que, sozinho, nosso explorador gasta cerca

Tabela 4: Bancada de atribuição de H5 (25 jogos públicos × 4 sementes × 1.500 passos; K sozinho). A RHAE está em pontos percentuais, como no *leaderboard*. Vitórias são níveis concluídos; vitórias e contagens são somadas sobre as sementes; o DP é o populacional sobre as quatro sementes; vitórias ambíguas têm mais de uma instância verdadeira. K0 mantém os guias H1/H2, que H5 substitui pela sua instância líder. «Sem G4» iguala a pontuação padrão com trajetórias diferentes, absorvidas pelo teto de 1,15.

braço	**RHAE ± DP**	**vitórias**	**sem inst. verdadeira**	**ambíguas**
K0 (sem H5; guias H1/H2)	0,3808 ± 0,1042	65	—	—
padrão (O3, G1, G2, G4)	0,3641 ± 0,0817	66	50	10
sem O3	0,3674 ± 0,1004	66	54	10
sem G1	0,4156 ± 0,0811	67	52	9
sem G2	0,3287 ± 0,1122	64	59	0
sem G4	0,3641 ± 0,0817	66	49	10
padrão + O1	0,3198 ± 0,0903	63	19	43
ordem aleatória	0,3557 ± 0,0952	63	48	9

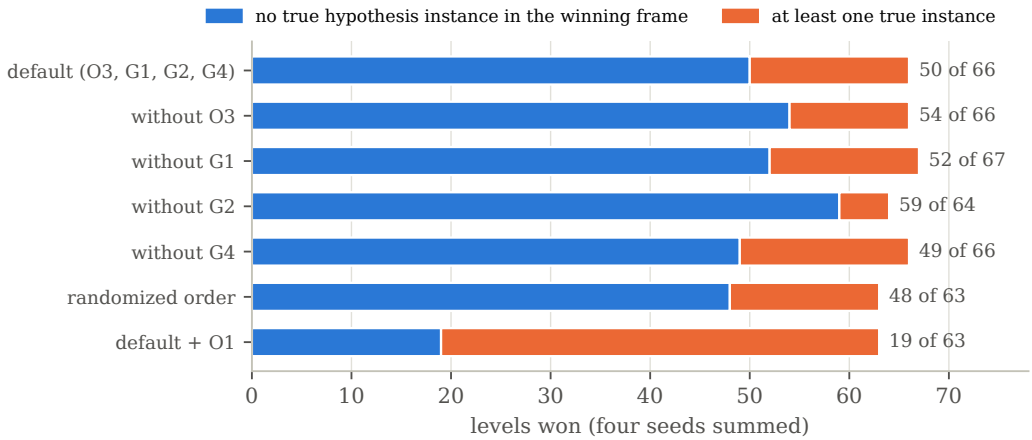


Figura 4: Bancada de atribuição de H5 (Tabela 4): níveis vencidos por braço, divididos conforme alguma instância de predicado implementado é verdadeira no quadro vencedor. «Verdadeiro» significa que um predicado que implementamos vale, não que ele é o objetivo do jogo; com O1, um predicado amplo de proximidade, a maioria das vitórias contém instâncias verdadeiras (43 delas ambíguas) sem ganho algum de pontuação. Os rótulos da figura estão em inglês, como no original.

de 13 vezes as ações humanas por nível concluído (Seção 5). Na nossa ablação, aconselhar não teve efeito mensurável.

O que não se transfere é o lance. Cada jogo fixa um objetivo e varia seus parâmetros de nível para nível, e nunca enuncia as regras que tornam alguns lances válidos e outros impossíveis. Nos termos de Ackoff e Emery, pela nossa leitura, os níveis de um jogo são ambientes estruturalmente diferentes para um objetivo, e jogos diferentes lidam com objetivos diferentes (1972/2017, pp. 30–31). Uma aresta é um fato sobre um jogo sob suas regras, e assim também um certificado e uma rota, razão pela qual a memória da Seção 3.4 carrega só uma família ou um esquema de papéis. Churchman (1971, pp. 125–126) imagina observadores que examinam partidas concluídas de jogo da velha «para detectar a característica terminal comum» e um matemático que representa o mesmo jogo como o preenchimento de uma matriz «de modo que ela satisfaça alguma propriedade». Quando os outros reclamam que ele «introduziu demais do seu próprio repertório», ele responde que eles também introduziram o deles, e que sua representação «pode ser usada para induzir regras para muitos outros jogos». Nossas famílias de objetivo desempenham papel semelhante, uma representação trazida do repertório do projetista. Onde o matemático reivindica esse alcance, não testamos o nosso além desta família de ambientes. A contagem de 50 de 66 é o que esse repertório deixa escapar aqui. O orçamento de vida de K é aprendido dentro de cada jogo, e a regra que o infere foi escolhida e testada só nos 25 jogos públicos, como as outras constantes de K e os tetos do último recurso.

Uma alavanca, a contagem de ações por nível, atravessa os três modos. Depois do prazo, no tópico do participante classificado em sexto, o classificado em quarto perguntou se a eficiência de ações, «não tokens por nível, mas ações por nível», era «uma alavanca que deixamos passar». O autor do tópico concordou, e o classificado em quarto relatou que incluir a fórmula da pontuação no prompt às vezes fazia o modelo «simplesmente desistir» (Zatravkin, 2026b). Nos ensaios da equipe classificada em sexto nos 25 jogos públicos, lances extras consumiram 15,2–20,1% dos pontos dos níveis vencidos ao fim de três execuções de 9 horas (11,8–16,1 pontos por cópia contra creditar cada vitória com, no pior caso, a contagem humana de ações). Nas mesmas execuções, 30–34% das vitórias usaram mais ações do que a linha de base humana (Zatravkin, 2026a). Nossos certificados agem sobre essa alavanca na lista de ações, mas, no jogo que diagnosticamos, os lances caros vieram do código do modelo, que contornava a lista (Seção 5). Não mostramos que nossas notas ou nossa lista mudaram as contagens de ações do modelo.

7 Limites e resultados negativos

- (i) O explorador, que não aprende modelo de mundo, estabiliza em cerca de 16 de 183 níveis públicos por semente e numa mediana de cerca de 13 vezes as ações humanas por nível concluído. Esse custo foi medido com o explorador sozinho, fora de qualquer base de modelo de linguagem.
- (ii) A identificação da condição de vitória, como implementada, é fraca onde importa, já que em 50 de 66 vitórias nenhuma instância de hipótese verdadeira está presente no quadro vencedor. Isso se assemelha a uma falha de segunda ordem no sentido de Bateson, escolher do conjunto errado de alternativas (1972, p. 286). O conjunto de testemunhas implementado não correspondeu ao quadro vencedor, e mudar só a ordenação não pode criar uma testemunha ausente.
- (iii) Nenhum braço de H5 mostra efeito mensurável na pontuação na bancada de atribuição, e as gerações anteriores mediram de levemente negativo a neutro. Lehman e Stanley relatam resultado semelhante num labirinto sem paredes externas, construído para testar os limites do

seu método. Ali, a busca por novidade encontrou o objetivo só cinco vezes em cem execuções, e a busca baseada em objetivo não se saiu melhor, com duas (2011, pp. 204–205). Do mesmo modo, substituir as heurísticas de objetivo fixas de K pelas famílias de H5 não fez a busca ter sucesso. Submetemos o componente pelo seu valor de medição e não fazemos afirmação de pontuação para ele.

- (iv) A marcação de estado oculto pelo gatilho (ii) disparou uma vez com zero certificados. Se marcar um nível como de estado oculto só com essa evidência é a resposta certa é uma questão de desenho aberta, que relatamos em vez de resolver.
- (v) O modo 1 compara bases diferentes e é evidência compatível com o custo das ações do explorador, não uma estimativa dele. Na mesma base, o modo 1 tem duas rodadas descritivas no primeiro lote e três no segundo, onde não mostra efeito mensurável contra um controle do mesmo dia (Tabela 3). O modo 2 não mostra efeito mensurável para o conselheiro em nenhum dos lotes, nem para o texto de aconselhamento sozinho no primeiro (três rodadas válidas por braço), embora toda rodada de texto de aconselhamento do primeiro lote fique abaixo da base. O modo 3 limita a intervenção por construção, mas não a perda de pontuação. Sua janela comparável foi medida na fase A no segundo lote, sem efeito mensurável, e sua única tiragem no conjunto oculto (28,41 contra 26,74, pontuada depois do prazo) está dentro da dispersão da base.
- (vi) A memória entre jogos foi submetida e não exercitada na fase A, e sua trajetória depende da temporização das *threads*.
- (vii) Os números do *leaderboard* carregam ruído de tiragem (DP 3,9 entre cópias inalteradas da base submetidas por outras equipes, média perto de 26), e cada submissão rende uma única tiragem no *leaderboard*. Deltas de *leaderboard* entre versões são portanto evidência fraca, e não argumentamos a partir deles. Nossas dispersões de fase A e de bancada são relatadas onde medidas. Pacotes inalterados resubmetidos pela equipe classificada em sexto tiraram 22,75/23,46 e 21,48/26,06 no *leaderboard* público (Zatravkin et al., 2026). Dois lotes da base sem modificação nos mesmos jogos públicos, com cinco dias de intervalo, diferiram em cerca de 23 pontos na média (74,91 contra 52,30), logo os níveis de fase A tampouco são comparáveis entre lotes.

8 Trabalho futuro: não jogar, não falar, entregar o mapa

O conselheiro ainda fala, já que poda uma lista e escreve notas num prompt. A configuração seguinte remove os dois canais e entrega o grafo de K à ferramenta do modelo como o `k_map` global. O mapa contém os estados, as arestas observadas com seus efeitos, `here`, `untried_here` e `route_to_untried`, o caminho **conhecido** mais curto (um limite superior no subgrafo observado, sem garantia de otimalidade global). Só arestas observadas entram, e arestas herdadas de fusões de estados ficam de fora. Em níveis de estado oculto, o mapa se declara inutilizável. É uma testemunha revogável (termo nosso) num nível de abstração declarado no sentido de Floridi, um conjunto finito de observáveis (2019, pp. 42–45), que aqui são o quadro mascarado e as ações testadas. Como um nível de abstração pode omitir os observáveis que tornariam as transições determinísticas (pp. 44–45), quadro $\neq$ estado é enunciado em vez de ser descartado por suposição. O mapa foi implementado e executado na fase A em 2 de novembro, numa versão que ainda deixava K jogar a janela de último recurso. Foi consultado uma vez em ls20 (depois que o modelo viu as ações de K no seu histórico) e não nos outros dois jogos, uso pequeno demais para avaliar seu valor. Neste *harness*, este modelo fez

pouco uso visível de qualquer dos canais, já que as notas ficaram sem menção no único jogo que diagnosticamos (Seção 5), e o mapa recebeu essa única consulta.

Uma variante posterior, só de leitura, sem a janela de último recurso, tornou o canal descobrível por uma linha fixa no prompt e um aviso curto no retorno da ferramenta, no máximo uma vez por nível, para uma chamada `k_read()` que nunca age. Ela não tinha poda nem notas do explorador, contava as chamadas executadas e rodou sob uma regra de validade fixada de antemão e uma liberação de *red team* independente. Numa rodada de fase A, sem braço de controle, foi consultada uma vez em três jogos. Essa consulta veio logo depois de um reinício e devolveu um instantâneo inutilizável, já que nenhuma observação correspondia ao quadro atual. Dois outros trechos em ls20 tinham telemetria incompleta e não foram contados como consultas. A rodada pontuou 42,70, abaixo das três rodadas da base da Seção 5, e, com uma rodada e sem controle, nem o canal nem seus avisos podem receber crédito pela diferença. Uma consulta mostra que o canal foi lido, e não que uma decisão mudou. Dzotsenidze (2026) traça a mesma linha para uma rota de recuperação que executou oito vezes: execução, ação seguinte, progresso e pontuação são observações separadas, e nenhuma substitui outra. Nosso resultado de baixo uso é específico destes canais e deste *harness*. Em outro agente baseado em dfranzen, uma requisição de 118.173 tokens pôs 25,0% das suas seleções de atenção esparsa em saídas de ferramenta (Zatravkin et al., 2026). Isso é um perfil de leitura, e não um perfil de uso.

Um antecedente próximo de entregar o registro ao código do modelo é o PRO-LONG (Fox et al., 2026, pp. 2, 4, 7–8). Seu *harness* anexa cada ação e tabuleiro a um registro que agentes de programação de fronteira pesquisam com código. Nas suas execuções com o GPT-5.5, 20,3% das chamadas de ferramenta além da saída de ação foram a esse registro, e acrescentar busca por expressão regular e depois código à leitura de arquivo elevou a pontuação (23,1 $\rightarrow$ 27,2 $\rightarrow$ 38,3). Esses são resultados no arranjo deles e não um teste pareado do nosso canal.

Para o canal do grafo em si, o laboratório arc-3 relata três execuções com grafo de estados de pontuação mais baixa apesar do uso intenso da ferramenta e de zero rejeições de plano. Uma variante enxuta expondo `untried_here` também pontuou mais baixo num A/B de sete jogos (1,533 com, 2,061 sem; sonpham-org, 2026). Os *harnesses* diferem, logo relatamos esse resultado ao lado do nosso sem opor os dois.

A fundação relata, do outro lado, a divisão que a Seção 5 mede para a exploração sozinha (cerca de 13 vezes as ações humanas por nível concluído). Antes do lançamento, ela havia hipotetizado que a eficiência de ações permaneceria uma linha divisória entre humanos e IA. Isso, escreve agora, «continua verdadeiro para abordagens de força bruta», e, uma vez que a IA de fronteira «entende» as mecânicas, ela «geralmente executa dentro da faixa da eficiência humana» (Kamradt, 2026b). A distinção é mais antiga do que o benchmark. Harlow descreveu os conjuntos de aprendizagem da Seção 3.4 como transformando um animal «de uma criatura que se adapta a um ambiente mutável por tentativa e erro numa que se adapta por aparente hipótese e *insight*» (1949, p. 51). Tolman, estudando como ratos aprendem labirintos, separou um mapa «em faixa», estreito, que liga posição a objetivo «por apenas um caminho relativamente simples e único», de um mapa abrangente do ambiente mais amplo. A diferença só aparece quando o ambiente muda, e quanto mais estreito o mapa, «menos ele se transferirá com sucesso ao novo problema» (1948, p. 193). Entre suas causas, ele listou «uma overdose de repetições no caminho originalmente treinado» (pp. 205, 207). Para eles, esses são estados de um organismo. Aqui eles descrevem o que o grafo do nosso explorador é, um registro das rotas de um nível, e motivam o primeiro dos dois braços abaixo, que pergunta o que se transfere ao nível k+1.

Dois braços adicionais, ainda não executados, seriam lidos pela regra da ablação (três rodadas por braço; duas vezes o maior DP dentro do braço). O primeiro é um registro de mecânicas entre níveis, motivado pela expectativa de que o primeiro nível de um jogo demonstra as mecânicas que seus níveis posteriores elaboram. É medido como ações até o primeiro progresso no nível k+1 com e sem o registro, uma taxa contada em tentativas com o objetivo mantido fixo. É assim que Ackoff e Emery medem a inteligência, como a taxa com que um sujeito aprende (1972/2017, pp. 52–53), aqui estendida a um sistema abaixo dos sujeitos com propósito para os quais eles a definem (pp. 34, 52). A medida inclui o lado negativo, o atraso quando a mecânica muda. Quando os problemas de Harlow passaram sem aviso de recompensar um objeto a recompensar uma posição, o conjunto de aprendizagem estabelecido dos macacos primeiro atrasou a nova aprendizagem (1949, pp. 60–61). Bateson prevê esse caso, já que, quando uma Aprendizagem II anterior identifica mal o padrão de contingência de um contexto novo, «devemos esperar um atraso da Aprendizagem I no novo contexto» (1972, p. 294). O braço se divide em ordenação de efeitos por objeto (M2a) e macroações reutilizadas (M2b), no sentido das opções de Sutton, Precup e Singh, políticas de malha fechada que, uma vez iniciadas, correm por vários passos até sua condição de parada (1999, pp. 181, 186). O segundo braço é a suspensão de família no componente de objetivo, uma revisão do conjunto de famílias (Aprendizagem II no sentido de Bateson, 1972, p. 293; Seção 3.4). Ela é decidida por um sinal de segunda ordem (crédito uniforme entre famílias, ou revisões crescendo sem ganhos de nível) e não pelo escalar de progresso de cada família. Suspensões e retornos são ambos contados, e a suspensão nunca apaga.

Três direções decorrem dos limites acima. A primeira diz respeito aos conectores, já que nossa camada alcançou o modelo só pelo prompt, pela lista de ações e, nas variantes posteriores, por um mapa consultável. No jogo que diagnosticamos, o código do modelo contornou a lista (Seção 5). Um *harness* construído para consumir tais afirmações, ou um modelo ajustado para lê-las, diria se as afirmações falham ou se só o seu canal falha. Não refinamos nem o modelo nem o *harness* para isso. A segunda diz respeito ao espaço de hipóteses, em que as famílias de objetivo foram autoradas por nós e 50 de 66 vitórias caíram fora delas. Famílias induzidas de transições de nível, na linha do EMPA e do agente Misfit (Seção 2), moveriam parte dessa autoria do desenvolvedor para o agente. A terceira diz respeito às sondas, já que a conjectura da Seção 1 pode ser testada nas transcrições que guardamos, contando os turnos em que o modelo enuncia hipóteses concorrentes e as ações que toma para distingui-las.

Encerramos com uma sugestão sobre desenho de benchmark, oferecida dentro dos limites de um único estudo. Os maiores ganhos relatados nesta competição vieram do modelo e do seu serviço de inferência, com o agente quase inalterado. Poda de especialistas, concorrência, carregamento mais rápido e correções de escalonador elevaram a pontuação de uma equipe no *leaderboard* de cerca de 26 para 33,68 e depois para 37,75 (Lam Dang, 2026), e o participante classificado em sexto chamou a otimização do serviço de «a MAIOR coisa a melhorar», e as mudanças de *harness* «morreram no ruído» (Zatravkin, 2026b). Fora da competição, a pontuação de um modelo de fronteira subiu de 62,7% para 99,9% com um *harness* construído sobre os recursos de gestão de contexto do seu provedor, a um custo relatado de cerca de US$ 19.000 para essa execução, contra US$ 26.000 para a execução com o *harness* padrão (Kamradt, 2026b). Esses resultados são legítimos, e, pela nossa leitura, ainda assim recompensam computação e engenharia de inferência. Dois relatos qualificam o próprio *leaderboard*. O código idêntico de uma equipe pontuou entre 43,18 e 53,27 nas suas rodadas, uma dispersão de 10,1 pontos, “so single draws cannot separate most changes” (Tanaka et al., 2026). A mesma equipe guarda as suas vitórias: como o placar pontua um jogo pela sua melhor partida, depois de um WIN o *harness* envia RESET e repete a sequência vencedora sem as jogadas exploratórias, o que elevou a sua média local em 110 jogos de 48,3 para 59,3. Nos comentários,

a equipe segunda colocada relata que, em 825 vitórias gravadas, a repetição elevou a pontuação média por jogo de cerca de 67 para 92, um ganho agregado "on the order of +12 points" "with no improvement to the agent's actual solving ability", e a deixou sem confirmação no gateway da competição (Tanaka et al., 2026). Uma vitória repetida é pontuada como partida, e a pontuação não a distingue de uma primeira. Os *checkpoints* quantizados que as duas bases servem foram preparados por terceiros (Seção 3.1), e a tentativa de uma equipe de adaptar o modelo por aprendizado por reforço rodou por cerca de dois dias numa GPU maior do que a da competição, sem uma execução longa o bastante para medir um efeito no jogo (Lam Dang, 2026). O primeiro lugar público relata a mesma direção levada até o fim com os recursos que ela exige: poda de especialistas e quantização INT3 recuperadas por destilação, uma pilha privada de *serving* numa RTX PRO 6000 para 110 sessões concorrentes e aprendizado por reforço em oito GPUs B200 voltado aos estágios que o modelo base não resolvia (Chen, 2026). O treino aumentou a resolução dos estágios difíceis, mas reduziu a eficiência de ações, com as execuções acima da contagem humana subindo de 16 % para 26 %, de modo que a submissão final manteve o modelo base nas primeiras seis horas e trocou para o modelo treinado nas três finais. É a forma do nosso modo de último recurso, alcançada pelo outro lado. Nós mantivemos a base e acrescentamos um explorador, que não treina nada; a entrada vencedora manteve a base e acrescentou um modelo treinado e o seu *serving*, que custam o hardware que a trilha dedicada, abaixo, fixaria para todos os participantes. Nada disso testa se um agente identifica um objetivo, muito menos se escolhe um. Boa parte disso também é ajustada ao ambiente de execução da competição (sua latência de carregamento do modelo e seu teto de tempo de parede), de modo que mais tokens por segundo compram mais inferência por jogo (Seção 2; Seção 5, lições de método). Essa é uma maneira válida de competir, mas o que ela mede é vazão num ambiente, ao passo que a eficiência que a definição de AGI da fundação nomeia é a eficiência de adquirir uma habilidade (ARC Prize Foundation, 2026, p. 4). Chollet separou a eficiência de informação que sua definição mede da eficiência de computação e de tempo, que listou como alternativas que uma definição poderia acrescentar (2019, p. 41). Dois sistemas poderiam diferir amplamente em tal otimização e ambos satisfazer essa definição, assim como modelos de linguagem de linhagens e velocidades diferentes são todos chamados de IA.

A pontuação, portanto, conduz os participantes aos recursos que tornam o jogo mais barato. A fundação pede ideias. Hooker (2021) chama de loteria do hardware (*hardware lottery*) quando uma ideia de pesquisa vence por ser compatível com o software e o hardware disponíveis, «não porque a ideia seja superior às direções de pesquisa alternativas» (p. 58), e pergunta quais fracassos «ainda não temos o hardware para ver como sucesso» (p. 63). O ARC-AGI-3 não se destina a medir «a quantidade de inteligência humana que entrou no projeto» de um sistema construído para ele (ARC Prize Foundation, 2026, p. 15), e a inovação aberta está entre as perguntas para seus próximos benchmarks (Kamradt, 2026b). Lemos a inovação aberta, neste contexto, como um agente produzindo um objetivo ou uma estratégia que seus projetistas não escreveram nele, e um benchmark pode abrir espaço para isso de duas maneiras. Primeiro, pode pontuar cada componente que nomeia separadamente da eficiência de ações. Para a definição de objetivos, o agente enunciaria a condição de vitória que persegue antes de alcançá-la, e o enunciado seria pontuado contra a condição real do jogo. Um identificador como o nosso, que não nomeou o objetivo em 50 de 66 vitórias locais, seria então medido diretamente, em vez de só por meio das ações. Segundo, pode fixar o modelo e seu serviço entre os participantes numa trilha dedicada, de modo que as diferenças de pontuação venham do agente e não da computação. Tal trilha não negaria a lição amarga de Sutton, a de que «métodos gerais que alavancam a computação são, em última análise, os mais eficazes, e por larga margem» (2019, parágrafo 1). Ela mediria o agente à parte dessa lição, em vez de proibi-la. Os agentes que Sutton pede são os «que podem descobrir como nós, e não os que contêm o que descobrimos»

(parágrafo 8), e uma pontuação para a definição de objetivos mediria essa descoberta. Dehghani et al. (2021) defendem o mesmo para um teto de orçamento de ajuste, que daria «comparações menos atadas ao orçamento computacional da entidade proponente e mais aos méritos dos métodos em si» (p. 16). Todo benchmark, como eles dizem, «faz uma declaração sobre o que percebe como importante» (p. 1). Um benchmark cuja pontuação estivesse alinhada a todas as dimensões que nomeia daria às ideias que ainda não elevam a pontuação de hoje uma medida própria.

9 Conclusão

O ARC-AGI-3 avalia a definição de objetivos; no entanto, cada jogo fixa sua condição de vitória, e a pontuação mede com que economia ela é alcançada. Na taxonomia de Ackoff e Emery, o passo que os agentes competidores dão é a identificação de um objetivo que o ambiente determina, o que os coloca, o nosso incluído, no máximo entre os sistemas buscadores de múltiplos objetivos. Pela nossa definição de trabalho, uma inteligência geral também selecionaria seus objetivos (Seção 1). Em 50 de 66 vitórias locais, nenhum dos predicados de objetivo que implementamos valia no quadro vencedor; ainda assim, o explorador venceu por busca (Seção 5). O resultado é consistente com o que outros desenhos relatam a partir de direções diferentes, como no Twin, em que a dinâmica é aprendida com muito mais confiabilidade do que o objetivo (Skoutnev et al., 2026, p. 28). Em outra entrada, um explorador com modelo direto aprendido não pontuou mais do que uma política aleatória nos jogos públicos, e o seu autor conclui que um preditor de atributos que aprende bem o movimento «ainda pode não ter a representação necessária para identificar um objetivo» (Shamim, 2026, pp. 11, 14). No DiG-bench, uma breve descrição das regras e da condição de vitória elevou as vitórias de um modelo de 18 para 69 de 70 jogos (Battleday et al., 2026, p. 6). Sem modelo de mundo e com uma medida diferente, nosso resultado aponta para o mesmo componente.

O segundo achado diz respeito ao alcance, já que nenhum dos três modos de presença (Seção 5) produziu ganho mensurável de pontuação. No jogo que diagnosticamos, os lances caros do modelo de linguagem vieram do seu código, e não das afirmações da camada (Seção 5). Identificar a condição de vitória e fazer essa identificação alcançar as ações é onde colocamos a parte aberta da definição de objetivos, um dos quatro componentes através dos quais o benchmark define a inteligência como eficiência. Escolher objetivos, a condição que nossa definição de trabalho acrescenta, está além do que este benchmark pede e além do que testamos.

10 Reprodutibilidade

O notebook público («What Exploration Can Ignore: Duck fork, K playing», `https://www.kaggle.com/code/hralmjc/what-exploration-can-ignore-duck-fork-k-playing`) é uma cópia limpa do código da nossa entrada elegível que carrega o agente (ref. 57852519, o *fork* do Duck com K jogando, modo 1). Suas células de código e arquivos embutidos são idênticos aos do kernel submetido depois da renomeação, conferidos por comparação de árvore sintática abstrata, com os registros internos removidos, e ele não foi repontuado no conjunto oculto. Os três modos na base dfranzen estão publicados como notebooks com o mesmo código congelado, a saber K jogando (`https://www.kaggle.com/code/hralmjc/what-exploration-can-ignore-dfranzen-k-playing`), a base sem modificação como controle (`https://www.kaggle.com/code/hralmjc/what-exploration-can-ignore-dfranzen-control`), K aconselhando (`https://www.kaggle.com/code/hralmjc/what-exploration-can-ignore-dfranzen-k-advising`) e último recurso (`https://www.kaggle.com/code/hralmjc/what-exploration-can-ignore-dfranzen-last-resort`), que diferem dos artefatos congelados só em

como localizam seus insumos no Kaggle, conforme declarado nos manifestos. O Kaggle montou os mesmos insumos sob dois arranjos diferentes para kernels diferentes deste conjunto, e os notebooks resolvem os dois e registram qual encontraram. O segundo lote da Seção 5 (Tabela 3) rodou esses quatro notebooks. Os kernels posteriores dfranzen + K, incluindo a v14.1 (modo de último recurso mais a memória entre jogos, cuja trajetória depende da temporização das *threads* e é reproduzível em código, não em execução), são identificados por artefatos congelados e manifestos SHA-256, como a submissão pontuada da linha Duck. Os números da bancada local vêm de um *harness* de bancada separado (25 jogos públicos, sem conexão, pontuado pela RHAE do toolkit) que não faz parte do notebook submetido. Um pacote de reprodutibilidade, publicado num repositório público no GitHub (`https://github.com/hralm/arc-agi-3-paper-track-2026`) junto com este artigo, contém o código do agente (K, H8 e H5, incluindo a memória entre jogos), o *harness* de bancada com os sinalizadores dos oito braços de atribuição (Tabela 4, Figura 4), a suíte de testes com os mutantes do protocolo (Seção 4), os manifestos SHA-256 dos artefatos congelados e os scripts que geram as figuras. Ele traz também os cinco notebooks acima, com os metadados de kernel e os módulos embutidos, e, de cada rodada da fase A aqui relatada (Tabelas 1 e 3, Figura 3), os arquivos de que os números foram lidos, o log do kernel, o resumo do *harness* e o registro do benchmark, com um script que recompõe a partir deles, ou da saída de uma nova rodada no Kaggle, as conferências de validade e as tabelas. Quem clonar o repositório tem todos os componentes citados neste artigo sem visitar cada página do Kaggle. As notas de trabalho das sessões de desenvolvimento, e as transcrições do modelo e os logs de requisições das rodadas, não são publicadas. O protocolo em si está descrito por inteiro na Seção 4.

Referências

Ackoff, R. L., & Emery, F. E. (2017). *On purposeful systems: An interdisciplinary analysis of individual and social behavior as a system of purposeful events*. Routledge. (Original work published 1972)

ARC Prize Foundation. (n.d.). *ARC Prize 2026* [Documentation]. Retrieved September 29, 2026, from `https://docs.arcprize.org/arc-prize-2026`

ARC Prize Foundation. (2026). *ARC-AGI-3: A new challenge for frontier agentic intelligence* (Version 2, arXiv:2603.24621). arXiv. `https://doi.org/10.48550/arXiv.2603.24621`

Bateson, G. (1972). The logical categories of learning and communication. In *Steps to an ecology of mind* (pp. 279–308). Ballantine Books. ISBN 0-345-24684-5.

Battleday, R. M., Sandbrink, K., Cullen-Drohan, J., Yan, Z., Muller, T., Maguire, C., Kubicek, A., Greenlee-Scott, F., Sumant, S., Dao, T., Schmidhuber, J., Valko, M., Tenenbaum, J., Griffiths, T. L., Kurth-Nelson, Z., & Whittington, J. C. R. (2026). *DiG-bench: Discovery in games* (Version 1, arXiv:2608.12593). arXiv. `https://doi.org/10.48550/arXiv.2608.12593`

Bessis, H., Cottaar, J., Pressman, I., Smit, D., Tešnar, M., & Viel, S. (2026a). *Tufa Labs duck harness [June 30 milestone winner]* [Kaggle notebook]. Kaggle. Retrieved September 30, 2026, from `https://www.kaggle.com/code/jeroencottaar/tufa-labs-duck-harness-june-30-milestone-winner`

Bessis, H., Cottaar, J., Pressman, I., Smit, D., Tešnar, M., & Viel, S. (2026b, July 1). *Tufa Labs' winning solution for ARC-AGI-3 Milestone 1* [Online forum post]. ARC Prize 2026 – ARC-AGI-

3, Kaggle. Retrieved November 9, 2026, from https://www.kaggle.com/competitions/arc-prize-2026-arc-agi-3/discussion/717133

Chen, Y.-C. (2026, November 4). *[ARC-AGI-3] Public 1st place solution: Quick overview* [Online forum post]. ARC Prize 2026 – ARC-AGI-3, Kaggle. Retrieved November 9, 2026, from https://www.kaggle.com/competitions/arc-prize-2026-arc-agi-3/discussion/755795

Chollet, F. (2019). *On the measure of intelligence* (Version 2, arXiv:1911.01547). arXiv. https://doi.org/10.48550/arXiv.1911.01547

Churchman, C. W. (1971). *The design of inquiring systems: Basic concepts of systems and organization*. Basic Books. ISBN 0-465-01608-1.

Courtis, D., Li, W., & Sanner, S. (2026). *OPINE-World: Programmatic world modeling with ontology-error-prioritized interactive exploration for ARC-AGI-3* (Version 3, arXiv:2607.01531). arXiv. https://doi.org/10.48550/arXiv.2607.01531

Dehghani, M., Tay, Y., Gritsenko, A. A., Zhao, Z., Houlsby, N., Diaz, F., Metzler, D., & Vinyals, O. (2021). *The benchmark lottery* (Version 1, arXiv:2107.07002). arXiv. https://doi.org/10.48550/arXiv.2107.07002

Dzotsenidze, T. (2026, November 2). *Recovery, task progress, and execution evidence in ARC-AGI-3* [Kaggle solution writeup]. ARC Prize 2026 – ARC-AGI-3, Kaggle. Retrieved November 4, 2026, from https://www.kaggle.com/competitions/arc-prize-2026-arc-agi-3/writeups/recovery-task-progress-and-execution-evidence-in

Ecoffet, A., Huizinga, J., Lehman, J., Stanley, K. O., & Clune, J. (2021). *Go-Explore: A new approach for hard-exploration problems* (Version 4, arXiv:1901.10995). arXiv. https://doi.org/10.48550/arXiv.1901.10995

Floridi, L. (2019). *The logic of information: A theory of philosophy as conceptual design*. Oxford University Press. ISBN 978-0-19-883363-5.

Fox, A., Wang, J., Rosu, P., & Dhingra, B. (2026). *PRO-LONG: Programmatic memory enables long-horizon reasoning* (Version 2, arXiv:2607.20064). arXiv. https://doi.org/10.48550/arXiv.2607.20064

Franzen, D. [dfranzen]. (2026a). *ARC-AGI-3 Milestone 2 solution* [Kaggle notebook]. Kaggle. Retrieved October 26, 2026, from https://www.kaggle.com/code/dfranzen/arc-agi-3-milestone-2-solution

Franzen, D. [dfranzen]. (2026b, September 30). *ARC-AGI-3 Milestone 2: Building on the Duck harness* [Kaggle project writeup]. Kaggle. Retrieved November 9, 2026, from https://www.kaggle.com/writeups/dfranzen/arc-agi-3-milestone-2

Gharajedaghi, J. (2011). *Systems thinking: Managing chaos and complexity: A platform for designing business architecture* (3rd ed.). Morgan Kaufmann.

Harlow, H. F. (1949). The formation of learning sets. *Psychological Review*, *56*(1), 51–65. https://doi.org/10.1037/h0062474

Hooker, S. (2021). The hardware lottery. *Communications of the ACM*, *64*(12), 58–65. https://doi.org/10.1145/3467017

Kamradt, G. (2026a, March 25). Announcing ARC-AGI-3. *ARC Prize Blog*. ARC Prize Foundation. Retrieved October 23, 2026, from https://arcprize.org/blog/arc-agi-3-launch

Kamradt, G. (2026b, September 3). OpenAI's GPT-6 Astra on ARC-AGI-3. *ARC Prize Blog*. ARC Prize Foundation. Retrieved November 4, 2026, from https://arcprize.org/blog/astra

Lam Dang. (2026, November 4). *More tokens! From public NB to LB 161th (missed 42th places by a few hours)* [Kaggle solution writeup]. ARC Prize 2026 – ARC-AGI-3, Kaggle. Retrieved November 4, 2026, from https://www.kaggle.com/competitions/arc-prize-2026-arc-agi-3/writeups/more-tokens-from-public-nb-to-lb-161th-missed-4

Le Grand, S. (2026). *taaf-flashnext-sheetu12b-0922* [Kaggle notebook; `agentfix` module]. Kaggle. Retrieved September 30, 2026, from https://www.kaggle.com/code/scottlegrand/taaf-flashnext-sheetu12b-0922

Lehman, J., & Stanley, K. O. (2011). Abandoning objectives: Evolution through the search for novelty alone. *Evolutionary Computation*, *19*(2), 189–223. https://doi.org/10.1162/EVCO_a_00025

Lehmann, J., Aioanei, A., & Vahdati, S. (2026). *Tycho: Active abstraction with programmatic world models for ARC-AGI-3* (Version 1, arXiv:2607.28287). arXiv. https://doi.org/10.48550/arXiv.2607.28287

Liew, K. H. (2026). *Explore before you solve: The speed-depth trade-off in epistemic agents for ARC-AGI-3* (Version 1, arXiv:2605.25931). arXiv. https://doi.org/10.48550/arXiv.2605.25931

Maturana, H. R., & Guiloff, G. D. (1980). The quest for the intelligence of intelligence. *Journal of Social and Biological Structures*, *3*(2), 135–148. https://doi.org/10.1016/0140-1750(80)90006-8

McCree, A. (2026). *AGI in a video shop - Atom Eons Nostalgia* [Kaggle notebook; Misfit agent]. Kaggle. Retrieved October 26, 2026, from https://www.kaggle.com/code/atommccree/agi-in-a-video-shop-atom-eons-nostalgia

Rodionov, S. (2026). *Executable world models for ARC-AGI-3 in the era of coding agents* (Version 2, arXiv:2605.05138). arXiv. https://doi.org/10.48550/arXiv.2605.05138

Roli, A., Jaeger, J., & Kauffman, S. A. (2022). How organisms come to know the world: Fundamental limits on artificial general intelligence. *Frontiers in Ecology and Evolution*, *9*, Article 806283. https://doi.org/10.3389/fevo.2021.806283

Rudakov, E., Shock, J., & Cowley, B. U. (2025). *Graph-based exploration for ARC-AGI-3 interactive reasoning tasks* (Version 1, arXiv:2512.24156). arXiv. https://doi.org/10.48550/arXiv.2512.24156

Sestak, F., Aichberger, L., Ielanskyi, M., & Klambauer, G. (2026, November 6). *[Public LB: 7] The last dance: Short overview* [Kaggle solution writeup]. ARC Prize 2026 – ARC-AGI-3, Kaggle. Retrieved November 9, 2026, from https://www.kaggle.com/competitions/arc-prize-2026-arc-agi-3/writeups/public-lb-7-short-overview

Shamim, A. (2026, November 6). *Think about AGI: A Duck based ARC AGI 3 agent and a study of adaptive action verification* [Solution paper, a project file of the Kaggle solution writeup "Think about AGI"]. ARC Prize 2026 – ARC-AGI-3, Kaggle. Retrieved November 9, 2026, from https://www.kaggle.com/competitions/arc-prize-2026-arc-agi-3/writeups/think-about-agi

Skoutnev, A., Acharya, K., Longhitano, G., Udell, M., Ellis, K., & Drori, I. (2026). *Twin: Playing an unknown game with a test-time digital twin* (Version 1, arXiv:2608.14490). arXiv. https://doi.org/10.48550/arXiv.2608.14490

sonpham-org. (2026). *arc-3: ARC-AGI-3 Duck Harness lab* [GitHub repository README; an instrumented fork of the Tufa Labs harness]. GitHub. Retrieved September 30, 2026, from https://github.com/sonpham-org/arc-3

Spelke, E. S., & Kinzler, K. D. (2007). Core knowledge. *Developmental Science*, *10*(1), 89–96. https://doi.org/10.1111/j.1467-7687.2007.00569.x

Sutton, R. S. (2019, March 13). *The bitter lesson.* Incomplete Ideas. http://www.incompleteideas.net/IncIdeas/BitterLesson.html

Sutton, R. S., Precup, D., & Singh, S. (1999). Between MDPs and semi-MDPs: A framework for temporal abstraction in reinforcement learning. *Artificial Intelligence*, *112*(1–2), 181–211. https://doi.org/10.1016/S0004-3702(99)00052-1

Taguchi, Y. [ ], shiiin9, Taiiiiiiichi, kTomoya, Sypetkowski, M., & Jaśkowska, N. (2026, November 4). *19th public LB place writeup* [Kaggle solution writeup]. ARC Prize 2026 – ARC-AGI-3, Kaggle. Retrieved November 9, 2026, from https://www.kaggle.com/competitions/arc-prize-2026-arc-agi-3/writeups/19th-public-lb-place-writeup

Tanaka Ai24, ISAKA Tsuyoshi, re, konbu17, pomcho555, yuto083, TK256, & t fuku. (2026, November 6). *[15th public] Tokens are progress: Serving engineering for an LLM coding agent* [Kaggle solution writeup, with comments]. ARC Prize 2026 – ARC-AGI-3, Kaggle. Retrieved November 9, 2026, from https://www.kaggle.com/competitions/arc-prize-2026-arc-agi-3/writeups/15th-public-tokens-are-progress-serving-enginee

Team Pandora [Pppycccc & pjh3316864904]. (2026, November 3). *33.39 and a bag of scars: MAXREQ tests, the 0.00 trap, and a REAP autopsy* [Kaggle solution writeup]. ARC Prize 2026 – ARC-AGI-3, Kaggle. Retrieved November 4, 2026, from https://www.kaggle.com/competitions/arc-prize-2026-arc-agi-3/writeups/33-39-and-a-bag-of-scars-maxreq-tests-the-0-00-t

Tolman, E. C. (1948). Cognitive maps in rats and men. *Psychological Review*, *55*(4), 189–208. https://doi.org/10.1037/h0061626

Tsividis, P. A., Loula, J., Burga, J., Foss, N., Campero, A., Pouncy, T., Gershman, S. J., & Tenenbaum, J. B. (2021). *Human-level reinforcement learning through theory-based modeling, exploration, and planning* (Version 1, arXiv:2107.12544). arXiv. https://doi.org/10.48550/arXiv.2107.12544

Zatravkin, G. (2026a, November 3). *ARC-AGI-3 run curves* [Public rehearsal dashboard; repository commit e5c4bb9]. GitHub Pages. Retrieved November 4, 2026, from https://gzatravkin.github.io/arc-agi-3-run-curves/

Zatravkin, G. (2026b, November 2). *Post competition discussion* [Online forum thread, with replies by other participants]. ARC Prize 2026 – ARC-AGI-3, Kaggle. Retrieved November 4, 2026, from `https://www.kaggle.com/competitions/arc-prize-2026-arc-agi-3/discussion/755202`

Zatravkin, G., Solange, F., _hans, & common-people. (2026, November 4). *6th public (54.55): Let the model tell you where the compute goes* [Kaggle solution writeup]. ARC Prize 2026 – ARC-AGI-3, Kaggle. Retrieved November 4, 2026, from `https://www.kaggle.com/competitions/arc-prize-2026-arc-agi-3/writeups/6th-public-54-55-let-the-model-tell-you-where-t`

Declaração

Este trabalho foi realizado com agentes de grandes modelos de linguagem, usados deliberadamente e sob um protocolo fixo, e não como geração sem revisão. O autor humano fixou as perguntas de pesquisa, as hipóteses, a definição de trabalho e o contraste teórico da Seção 1, e as regras sob as quais os agentes trabalharam: um pré-registro com condições de falseamento antes de cada submissão, um *blue team* antes de todo *red team*, um *red team* executado por um modelo diferente do que construiu o artefato, um teste de regressão e um mutante para cada defeito, a leitura integral de uma fonte antes de citá-la e, para o texto final, revisão bloco a bloco com a fidelidade às fontes conferida antes do estilo. Os agentes executaram o trabalho executivo: implementaram o código, prepararam os experimentos, rodaram os testes e as bancadas locais, leram as fontes e redigiram o texto como paráfrase dos argumentos do autor e das obras citadas. Também propuseram formulações e opções técnicas. Toda proposta e toda saída foi adjudicada pelo autor, que a aceitou, rejeitou ou redirecionou, bloco a bloco, à medida que o texto foi escrito e revisto. O autor executou todos os experimentos no Kaggle aqui relatados. Modelos de provedores diferentes e de perfis diferentes, ocidentais e orientais, ocuparam os papéis de *blue team* e de *red team*. O autor assume a responsabilidade pelo conteúdo; nenhum modelo de linguagem é autor deste trabalho.

Este documento é uma tradução do original em inglês, produzida com o auxílio de agentes de modelos de linguagem e lida pelo autor. As figuras mantêm os rótulos em inglês, e as referências são as do original. Onde as versões diferirem, a versão em inglês é a que vale.